Software Requirements Specification

for

Forkful: Cookbook Recipe Manager

Version 1.0 approved

Prepared by Melissa Weitekamp & Kel Knight

Grandma’s Recipe

10/14/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents **2**
Revision History **3**
1. Introduction **4**
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 5
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 6
2.3 User Classes and Characteristics 7
2.4 Constraints 7
2.5 Operating Environment 8
2.6 Design and Implementation Constraints 8
2.7 User Documentation 8
2.8 Assumptions and Dependencies 8
3. External Interface Requirements **9**
3.1 User Interfaces 9
3.2 Hardware Interfaces 9
3.3 Software Interfaces 9
3.4 Communications Interfaces 10
4. System Features **11**
4.1 Public Catalog Browsing and Search 11
4.2 Save and Edit a Copy of a Public Recipe 12
4.3 Add a Private Recipe Manually 14
4.4 Scan a Recipe via OCR 15
4.5 Delete Recipes 17
4.6 Admin Account Management 18
4.7 User Profiles 19
4.8 Account Creation 20
4.9 Login and Authentication 21
4.10 Library View and Organization 22
4.11 Cooking Mode 23
4.12 Catalog Curation 24
4.13 Parse a Recipe from a URL 25
5. Other Nonfunctional Requirements **26**
5.1 Performance Requirements 26
5.2 Safety Requirements 27
5.3 Security Requirements 27
5.4 Software Quality Attributes 28

5.5 Business Rules 29
6. Other Requirements 30
6.1 Database and OCR Engine Decisions 30
6.2 Data Requirements 32
6.3 Data Retention 32
6.4 Legal and Localization 33
Appendix A: Glossary 33
Appendix B: Analysis Models 34
Appendix C: To Be Determined List 42
Appendix D: Elicitation, Validation, AI Disclosure 43
D.1: Elicitation Log 43
D.2: Validation 44
D.3: AI Disclosure (H/A/X) 45
Appendix E: Potential Future Features 45

Revision History

Name	Date	Reason For Changes	Version
All group members	10/9/2026	Initial doc changes	1.0
Melissa W.	10/27/2026	Added use cases UC-3 to UC-13 and features 4.9 to 4.13	1.1
All group members	11/1/2026	Added NFR-6 to NFR-31, BR-4 to BR-12, and §3.6 and §6	1.2
All group members	11/4/2026	Chose PostgreSQL 18 and Tesseract 5.5; fixed UC-2, NFR, catalog-location and saved-copy inconsistencies	1.3
Melissa W.	11/5/2026	Redrew Figure B-1 (use cases) and Figure B-3 (data flow); added Figures B-5 to B-11	1.4
All group members	11/5/2026	Scoped features for ease of hosting, moved deferred features into new Appendix E: Potential Future Features	1.5

1. Introduction

1.1 Purpose

This SRS specifies the requirements for the Cookbook Recipe Manager (Forkful), version 1.0. Forkful is a web application that lets home cooks save recipes from a public catalog, modify saved copies without altering the originals, add or scan private recipes, and remove these recipes at any time. This document covers the v1.0 product. It does not cover the source blogs or the parser that populates the catalog

1.2 Document Conventions

Standards and typographical conventions in this SRS:

- Shall/must indicate mandatory requirements.
- "Original recipe" refers to the public catalog parsed recipe.
- "Saved copy" refers to a user's private copy/duplicate of a recipe that can be modified.
- "Library" refers to a user's private library of saved and created recipes.
- All requirements are uniquely identified: REQ-x.x for function, NFR-xx for non-functional requirements, UC-x for use cases, and BR-x for business rules.
- Features considered for a later version are listed in Appendix E. They are not part of this build and have no REQ or NFR numbers.

1.3 Intended Audience and Reading Suggestions

Reader	***Suggested Reading***
Developer	*§2, §3, §4, §5*
Tester	*§4, §5, Appendix B*
Project Manager	*§1, §2, §3.5*
Client / Product owner	*§1.4, §2.2, §4*
Maintainers	*§2.4, §2.6, §5*

1.4 Product Scope

Forkful is a web-based personal recipe manager. It lets users browse and search a public catalog of recipes populated by parsing recipes from blogs and cooking sites; save a copy of any public recipe and edit that copy while the catalog original stays unmodified; enter recipes manually or scan printed recipes via OCR, stored privately in the user's own library; and delete any saved copy or private recipes at any time.

Forkful will not publish or display any user's private recipes or host the source blogs and cooking sites from which the recipes are parsed.

Forkful's main benefit is a centralized personal cookbook that supports non-destructive recipe edits and digitization of paper recipes.

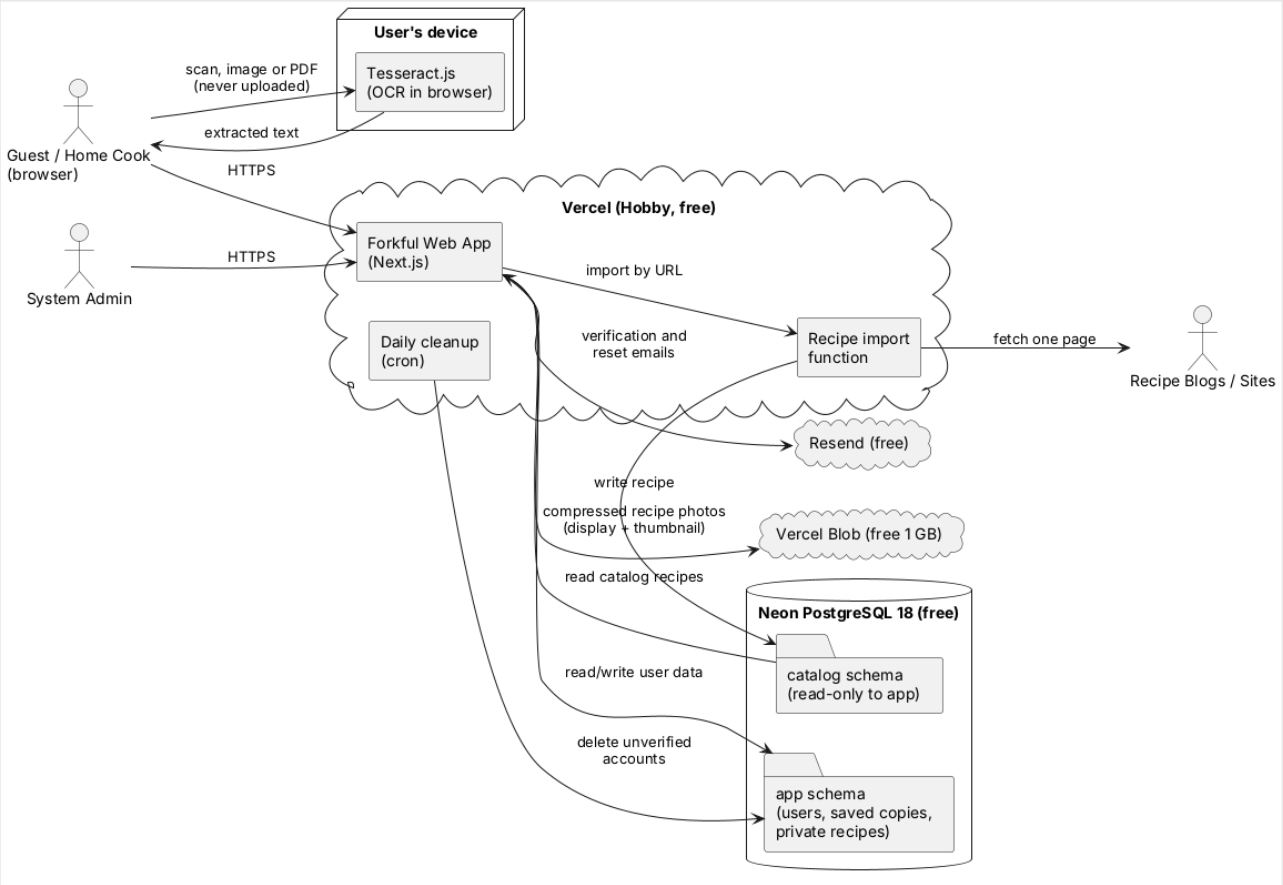
1.5 References

Weigers, K. *Software Requirements Template.* 1999.
Course Slide Deck. *CS445_Ch4_Requirements_Engineering_Complete.pptx.*
Forkful team. Adding/Scanning a Recipe Activity Diagram, Data Flow Diagram, and Save-to-Library Sequence Diagram. 2026 (Appendix B).
CS445 Architecture and Design Milestone Worksheet, Forkful. 2026.
PostgreSQL Global Development Group. PostgreSQL 18 Documentation. 2025.
Tesseract OCR project. Tesseract 5.5 Documentation.
Vercel, Neon, and Resent. Free plan documentation, checked November 2026 (§6.5).

2. Overall Description

2.1 Product Perspective

Forkful is a self-contained web application that runs on free service tiers (§6.1). It replaces the practice of saving recipes as loose paper sheets and browsing bookmarks by storing them digitally in one place. Itkeeps its own public recipe catalog, which grows when an admin or curator imports a recipe page by its URL, and it reads scanned recipes with Tesseract.js, an open-source OCR library that runs in the user's browser.



```
@startuml F21_system_context
title Figure 2-1. Forkful System Context
skinparam shadowing false
skinparam defaultFontName Arial
skinparam componentStyle rectangle
left to right direction
actor "Guest / Home Cook\n(browser)" as Browser
actor "System Admin" as Staff
actor "Recipe Blogs / Sites" as Blogs
node "User's device" {
  component "Tesseract.js\n(OCR in browser)" as OCR
}
cloud "Vercel (Hobby, free)" {
  component "Forkful Web App\n(Next.js)" as App
  component "Recipe import\nfunction" as Import
  component "Daily cleanup\n(cron)" as Cron
}
database "Neon PostgreSQL 18 (free)" {
  folder "catalog schema\n(read-only to app)" as Catalog
  folder "app schema\n(users, saved copies,\nprivate recipes)" as AppSchema
}
cloud "Resend (free)" as Mail
cloud "Vercel Blob (free 1 GB)" as Blob
Browser --> App : HTTPS
Browser --> OCR : scan, image or PDF\n(never uploaded)
OCR --> Browser : extracted text
Staff --> App : HTTPS
App --> Import : import by URL
Import --> Blogs : fetch one page
Import --> Catalog : write recipe
Catalog --> App : read catalog recipes
App <--> AppSchema : read/write user data
Cron --> AppSchema : delete unverified\naccounts
App --> Mail : verification and\nreset emails
App --> Blob : compressed recipe photos\n(display + thumbnail)
@enduml
```

2.2 Product Functions

The Forkful web application software offers:

- Account registration, login, and profile management to access full features of the website
- Search features to sort and filter the public recipe catalog
- Scale servings and convert units (US or metric) on any recipe
- Save a copy of a public recipe to the user's personal library
- Create a private recipe manually
- Scan a printed recipe via OCR and review/correct the result
- Organize recipes in the Library with tags
- Delete saved or private recipes from the user's personal library
- Cooking mode with a step-by-step view that keeps the screen awake

- Delete the account
- Admin console for accounts and roles, and adding a recipe to the catalog from its URL
- Catalog curation by the System Admin
- Add an optional dish photo to a recipe, compressed for fast display in the catalog and Library

2.3 User Classes and Characteristics

Stakeholders	Frequency Use	Skill	Privilege Levels	Role
Casual Browser (guest)	Occasional	Low-technical skill	Read-only, no account	Unauthenticated visitor who browses the public catalog
Home Cook (registered user)	Daily-weekly	Low-moderate skill	Full access to own library only	Authenticated home cook who saves, edits, scans, and deletes recipes.
System Administrator	As needed	High skill	Highest privilege	Manages accounts, roles, and catalog imports.

What each stakeholder needs from the system:

- Casual Browser - Browse and search the public catalog without registering.
- Home Cook – Save a copy of a public recipe and edit it without altering the original, keep scanned and personal recipes private, delete recipes at any time.
- System Admin – Reliable backups on the database, role-based access control, and ability to pause/resume parser and OCR jobs.

2.4 Constraints

Forkful shall abide by the following legal constraints:

- Children's Online Privacy Protection Act (COPPA)
- Terms of service from source sites
- Consumer privacy as defined:
    - Allow users to query their personal information, as collected.
    - Allow users to delete their account and all information attached to it.
    - Keep private recipes hidden and unable from appearing in public recipes and searches.

Forkful shall also abide by the following technical constraints:

- Must run on modern browsers and their latest versions.
- OCR accuracy depending on the OCR engine used.
- Must be written with the programming languages and frameworks latest version.

Saving speed and local draft caching are specified in NFR-4 and NFR-9 (§5*)*.

Data stored by Forkful must abide by the following constraints:

- Scans must be JPG, PNG, or PDF files (up to 5 PDF pages). They are read in the browser only while a recipe is being added, and are never uploaded or stored.
- Scans and dish photos must be no more than 10 MB before compression.
- Each user can store up to 100 recipes, saved copies plus private recipes (BR-10).
- Forkful stores one compressed photo per recipe: a display version of at most 100 KB and a thumbnail of at most 20 KB, both WebP (NFR-8, §6.1).

2.5 Operating Environment

Forkful is expected to work these modern browsers for both desktop and mobile devices:

- Google Chrome
- Apple Safari
- Mozilla Firefox

2.6 Design and Implementation Constraints

Possible design and implementation constraints for Forkful are listed below:

- Forkful's public catalog is read-only from the user's perspective; save copies must be separate records for modifications.
- Recipe import must respect each source site's robots.txt and terms (BR-3) and fetch one page per request.
- All user data should be encrypted in transit and at rest (NFR-3).
- The System Admin reviews failed imports and flagged catalog entries at least once a week.
- Data is stored in PostgreSQL 18 on Neon's free plan using two schemas: catalog (read-only to the app) and app (users, saved copies, private recipes). See §6.1.
- Saved copies are stored as full, independent recipe records linked to their source by catalog ID, not as override records (§6.1).
- OCR uses Tesseract.js in the user's browser, after PDF pages are turned into images with PDF.js; scans never leave the user's device.
- Forkful is a Next.js (TypeScript) app on Vercel's free Hobby plan, which allows personal, non-commercial use only (BR-11).
- Constraints listed in §2.4

2.7 User Documentation

Forkful will feature a web-based video tutorial to help users and guests explore Forkful's features. It will have a support page with multiple FAQ and how-to tutorials as well. An Admin Guide PDF will provide information on the catalog and parser and OCR configuration.

2.8 Assumptions and Dependencies

These are possible assumptions and dependencies for Forkful:

- The public catalog is populated when the System Admin imports a recipe page by URL. Forkful reads it through its internal Catalog Service (CatalogQueryAPI).
- Users have a device that can photograph a recipe and run OCR in a supported browser.
- Users will not try to use Forkful on an unsupported browser.

- Vercel, Neon, and Resend keep their free plans at or above the limits used in §6.1.
- Users signing up for Forkful will have an email address to create their account.
- Tesseract.js remains open source and runs in current versions of the supported browsers.
- If a recipe import fails, the catalog does not grow, but the rest of Forkful keeps working.
- Forkful runs entirely on free service tiers (§6.1). If a provider lowers its free limits, the team will move to another free option in §6.5 or lower the limit in BR-10.
- The free plans in §6.1 have no end date and need no card, so Forkful can stay online after the class ends without depending on student credits.

3. External Interface Requirements

3.1 User Interfaces

Forkful shall have a responsive web UI that is mobile-friendly. It will have two modes: Browser mode UI and Cooking mode UI.

- Browser mode shall remain as the default until Cooking mode is enabled.
- Cooking mode UI uses a digital “lock” to prevent the device screen from dimming and sleeping for until the recipe instructions are finished, the device is powered off, browser tabs switched, or if more than an hour has passed with Cooking mode (§4.13).

Buttons that are included on every page of Forkful:

- Home
- Search
- Library
- Profile
- Help

Error messages will use standard formatting: problem + cause + suggested action/solution

The OCR review screen shall be capable of showing the scanned image alongside the editable extracted text.

3.2 Hardware Interfaces

Optional camera/scanner input for OCR using USB or a mobile camera.

3.3 Software Interfaces

Forkful software relies on a web browser to properly fetch and display data within the Forkful system.

Other software interfaces used in Forkful include:

Component	Version	Direction	Purpose
Catalog Service (CatalogQueryAPI)	v1 (internal)	In	Fetch catalog recipes for browsing/search
Tesseract.js	Latest stable	Runs in browser	Convert scanned image to text on the user’s device

PDF.js	Latest stable	Runs in browser	Turn up to 5 PDF pages into images before OCR
PostgreSQL on Neon (free plan)	18	In/Out	Catalog schema (read-only to the app) and app schema
Vercel Functions (Hobby plan)	-	In/Out	Run the web app, API, recipe import, and the daily cleanup job
Email service (Resend)	-	Out	Verification and password-reset emails over an HTTPS API
Vercel Blob (Hobby plan)	-	In/Out	Store compressed recipe photos (display version and thumbnail)

3.4 Communications Interfaces

Forkful client software interface runs in a web browser and communicates via HTTPs protocol. An internet connection speed of 2.5Mbps – 5Mbps is required to use the full features of Forkful.

Recipe import fetches source pages over HTTPS, one page per request, and identifies itself as Forkful.

3.5 Operations

Nightly database backup at 2:00 AM by a free scheduled GitHub Actions workflow, plus Neon's 6-hour instant restore; recovery point objective (RPO) of 24 hours and recovery time objective (RTO) of 4 hours (NFR-26). The last 7 nightly backups are kept.

A daily Vercel cron job deletes accounts never verified after 7 days.

3.6 Functional Hierarchy

Forkful's functions grouped by user class. Section numbers refer to §4.

- **Casual Browser (guest)**
  - Browse and search the catalog (4.1)
  - Scale servings and convert units on any recipe (4.1)
  - Create an account (4.8)
  - Cooking mode on catalog recipes (4.11)
- **Home Cook (registered user)**
  - Log in, log out, reset password (4.9)
  - Save and edit a copy of a public recipe (4.2)
  - Add a private recipe manually (4.3)
  - Scan a recipe via OCR (4.4)
  - Delete and restore recipes (4.5)
  - View and organize the Library (4.10)
  - Cooking mode (4.11)
  - Manage profile, download data, delete account (4.7)
- **Content Curator**
  - Review and correct catalog entries (4.12)

- **System Administrator**
  - Manage accounts and parser/OCR jobs (4.6)
  - Parse a recipe from a URL (4.13)
  - Review and correct catalog entries (4.12)

4. System Features

4.1 Public Catalog Browsing and Search

4.1.1 Description and Priority

- Allow visitors and users to browser and search the public catalog of parsed recipes.
- Priority: High

4.1.2 Stimulus/Response Sequences

1. User opens Catalog -> system shows paginated recipes.
2. User enters a query -> system returns matching recipes.
3. User selects a recipe -> system shows details with “Save a Copy.”

Table 4.1.1 UC-1 Tabular Specification

Use Case	UC-1: Browse and Search the Public Catalog
Actor	Casual Browser (guest) or Home Cook (registered user)
Precondition	The public catalog contains at least one recipe. No login is required.
Main Flow	1. The user opens the Catalog page 2. The system displays the first page of 20 catalog recipes. 3. The user enters a search query and optionally selects cuisine or tag filters and a sort order. 4. The system returns matching recipes ranked by relevance within the time limit in NFR-1. 5. The user selects a recipe. 6. The system displays the recipe detail page with title, ingredients, steps, source site link, and a “Save a Copy” button.
Alternative Flows	A1 (no results, step 4): The system shows “No recipes match your search” and suggests removing filters or changing the query. A2 (catalog unavailable, step 2 or 4): The system displays an error message problem, cause, suggested action, and lets the user retry. A3 (recipe removed, step 6): The system shows “Recipe not found/unavailable” and returns to the results. A4 (change servings or units, step 6): The system recalculates the quantities on screen (REQ-1.11, REQ-1.15); the stored recipe is unchanged.
Postcondition	The catalog is unchanged, and the user has viewed search results or a recipe’s details.

4.1.3 Functional Requirements

- REQ-1.1: Display public catalog recipes in a paginated list of 20 per page. [Source: Interview (AI stakeholder)]
- REQ-1.2: Rank search results by relevance to the query and return them within the time limit in NFR-1. [Source: Interview (AI stakeholder)]
- REQ-1.3: Allow visitors to filter catalog results by cuisine and tags. [Source: Team]
- REQ-1.4: Display each catalog recipe's title and its source site. [Source: Team]
- REQ-1.5: Display a recipe detail page with the recipe's compressed photo (if it has one), title, ingredients, steps, source site name and link, and a "Save a Copy" button. [Source: Interview (AI stakeholder)]
- REQ-1.6: Allow users to sort catalog results by relevance, newest, and title (A-Z). [Source: Team]
- REQ-1.7: Show a "No recipes match your search" message with a suggestion to remove filters when a search returns no results. [Source: Team]
- REQ-1.8: Exclude private recipes and saved copies from all catalog results (BR-2). [Source: Team]
- REQ-1.9: Show each recipe's thumbnail in catalog results; a recipe without a photo shows a placeholder image. [Source: Team]
- REQ-1.10: Load the starting catalog of 100 to 500 recipes from a seed file the team prepares, using photos the team owns or that are freely licensed (BR-12). [Source: Team]
- REQ-1.11: Show a servings control on every recipe detail page, in the catalog and the Library, that recalculates every ingredient quantity on screen without changing the stored recipe. [Source: Team]
- REQ-1.12: Show scaled US amounts as common kitchen fractions (1/8, 1/4, 1/3, 1/2, 2/3, 3/4) and scaled metric amounts as whole grams or milliliters (one decimal place below 10). [Source: Team]
- REQ-1.13: Leave ingredient lines without a quantity unchanged, and round countable items to the nearest whole number with a note that the amount was rounded. [Source: Team]
- REQ-1.14: Allow a logged-in user to keep a scaled version: as a new saved copy for a catalog recipe, or by updating their own recipe after confirmation. A new copy counts toward the 100-recipe limit (BR-10). [Source: Team]
- REQ-1.15: Provide a US/metric toggled on every recipe detail page that converts quantities for display only; logged-in users start from their profile preference (REQ-7.8). [Source: Team]
- REQ-1.16: Convert within the same kind of measure only: tsp, tbsp, fl oz, cup, pint, quart, and gallon to and from mL and L; oz and lb to and from g and kg; and °F to and from °C for oven temperatures written as a number followed by °F or °C in the steps. Units with no conversion, such as pinch, clove, or can, stay as entered. [Source: Team]

4.2 Save and Edit a Copy of a Public Recipe

4.2.1 Description and Priority

- Lets registered users save a personal copy of a public catalog recipe and edit that copy without altering the catalog original.
- Priority: High

4.2.2 Stimulus/Response Sequences

1. User clicks “Save a Copy” on a catalog recipe.
2. System creates a new private record owned by the user and confirms.
3. Subsequent edits affect only the copy.

Table 4.2.1 UC-2 Tabular Specification

Use Case	UC-2: Save and Edit a Copy of a Public Recipe
Actor	Home Cook (registered user)
Precondition	The Home Cook is logged in, and the recipe exists in the public catalog
Main Flow	1. The Home Cook opens a public catalog recipe’s detail page. 2. The Home Cook clicks “Save a Copy.” 3. The system creates a new recipe record owned by the Home Cook, leaves the catalog original unchanged, and confirms the save. 4. The Home Cook opens the saved copy in their Library and edits one or more fields. 5. The Home Cook saves the changes 6. The system stores the edits on the copy only and displays a “Modified” indicator on it.
Alternative Flows	A1 (not logged in, step 2): The system prompts the user to log in or create an account, then returns to step 2. A2 (save fails, step 3): The system displays an error message (problem, cause, suggested action) and creates no record; the Home Cook may retry. A3 (edit cancelled, step 5): The system discards the unsaved edits and the saved copy is unchanged. A4 (recipe no longer in the catalog, step 3): The system shows “Recipe not found/unavailable” and creates no record. A5 (copy already saved, step 3): The system tells the Home Cook a copy already exists and offers to open it or save another copy. A6 (Library full, step 3): The system creates no record and shows “Your Library is full (100 of 100 recipes)” with a link to the Library.
Postcondition	A private copy exists in the Home Cook’s Library with any saved edits, and the catalog original is unmodified.

4.2.3 Functional Requirements

- REQ-2.1: Create a new recipe record owned by the user when saving a copy, leaving the catalog record unmodified. [Source: Team]
- REQ-2.2: Allow the user to edit any field of a saved copy. [Source: Team]
- REQ-2.3: Display a “Modified” indicator on any saved copy whose fields differ from the catalog original. [Source: Team]
- REQ-2.4: If the catalog recipe is no longer available when “Save a Copy” is clicked, show “Recipe not found/unavailable” and create no record. [Source: Sequence diagram]
- REQ-2.5: If the user already has a copy of the same catalog recipe, tell them and offer to open the existing copy or save another copy. [Source: Team]

- REQ-2.6: Keep saved copies when the catalog original is changed or removed, and show “Original updated” or Original no longer available” on the copy. [Source: Team]
- REQ-2.7: Block “Save a Copy” when the user already has 100 recipes in their Library, and show “Your Library is full (100 of 100 recipes)” and suggest deleting recipes. [Source: Team]
- REQ-2.8: Show the catalog recipe’s photo on a new saved copy, and let the user replace it with their own dish photo. [Source: Team]

4.3 Add a Private Recipe Manually

4.3.1 Description and Priority

- Lets a registered user enter a recipe from scratch and keep it private to their own library.
- Priority: High

4.3.2 Stimulus/Response Sequences

1. User selects “New Private Recipe.”
2. User enters fields and saves.
3. System stores the recipe as private to that user.

Table 4.3.1 UC-3 Tabular Specification

Use Case	UC-3: Add a Private Recipe Manually
Actor	Home Cook (registered user)
Precondition	The Home Cook is logged in and is viewing their Library.
Main Flow	1. The Home Cook selects “New Private Recipe.” 2. The Home Cook chooses Manual Entry. 3. The system shows a blank recipe form with the required title, ingredients, and steps fields and optional fields. 4. The Home Cook enters the recipe details and clicks Save. 5. The system checks that the required fields are filled. 6. The system stores the recipe as private, owned by the Home Cook, showed it in the Library, and confirms the save.
Alternative Flows	A1 (missing required field, step 5): The system highlights each missing field, shows a recipe error message, keeps the entered data, and returns to step 4. A2 (cancel, step 4): If there are unsaved entries, the system asks for confirmation; on confirmation it discards them and creates no recipe. A3 (save fails, step 6): The system displays an error message (problem, cause, suggested action), keeps the draft in the local cache (NFR-11), and the Home Cook may retry. A4 (Library full, step 1): The system shows “Your Library is full (100 of 100 recipes)” and does not open the form.
Postcondition	A new private recipe, visible only to its owner, exists in the Home Cook’s Library.

4.3.3 Functional Requirements

- REQ-3.1: Allow a logged-in user to create a private recipe with title, ingredients, steps, and optional tags. [Source: Team]
- REQ-3.2: Make every private recipe visible only to its owner. [Source: Team]
- REQ-3.3: Require a title, at least one ingredient, and at least one step before saving, and highlight each missing field. [Source: Activity diagram]
- REQ-3.4: Allow optional fields: description, servings, prep time, cook time, cuisine, photo, and notes. [Source: Team]
- REQ-3.5: Ask for confirmation before discarding unsaved entries when the user cancels. [Source: Team]
- REQ-3.6: Block "New Private Recipe" when the user already has 100 recipes, and show "Your Library is full (100 of 100 recipes)" and suggest deleting recipes. [Source: Team]
- REQ-3.7: Accept a JPG or PNG dish photo up to 10 MB, and compress it in the browser before upload into a display version (at most 1,200 px wide and 100 KB) and a thumbnail (at most 400 px wide and 20 KB), both WebP. [Source: Team]
- REQ-3.8: Delete a recipe's stored photo when the recipe is permanently deleted or the photo is replaced. [Source: Team]
- REQ-3.9: Store each ingredient line as a quantity, unit, name, and note; the form gives separate quantity and unit fields, with the unit chosen from a list. [Source: Team]

4.4 Scan a Recipe via OCR

4.4.1 Description and Priority

- Lets a registered user digitize a printed or handwritten recipe by uploading or photographing it, running OCR, correcting the extracted text, and saving the results to a private recipe.
- Priority: High

4.4.2 Stimulus/Response Sequences

1. User uploads/photographs a recipe image.
2. System runs OCR and shows extracted text beside the image.
3. User corrects text and saves as a private recipe.

Table 4.4.1 UC-4 Tabular Specification

Use Case	UC-4: Scan a Recipe via OCR
Actor	Primary: Home Cook (registered user) Supporting: OCR Engine (Tesseract)
Precondition	The Home Cook is logged in and has a recipe image (JPG, PNG, or PDF, up to 10 MB) to upload or photograph.
Main Flow	1. The Home Cook selects "New Private Recipe" and chooses Scan Entry. 2. The Home Cook uploads or photographs the recipe image. 3. The system checks the file type, size, and readability, then submits the image to the OCR Engine 4. The OCR Engine returns the extracted text. 5. The system shows a recipe preview: the extracted text split into title, ingredients, and steps fields, beside the source image.

	6. The Home Cook corrects the text and saves. 7. The system checks the required fields, stores the recipe as private and owned by the Home Cook, and confirms the save.
Alternative Flows	A1 (not logged in, step 2): The system prompts the user to log in or create an account, then returns to step 1. A2 (invalid file, step 3): The system shows an image upload error stating the allowed file types and size, and returns to step 2. A3 (OCR fails or takes longer than 30 seconds, step 4): The system shows an error and lets the Home Cook retry or switch to manual entry with the image kept for reference. A4 (missing required field, step 7): The system highlights each missing field and returns to step 6. A5 (cancel, step 6): The system discards the extracted text and the uploaded image; no recipe is created A6 (Library full, step 1): The system shows "Your Library is full (100 of 100 recipes)" and does not start a scan.
Postcondition	A private recipe with the corrected text, owned by the Home Cook, exists in their Library. The scan was never uploaded; only the optional compressed dish photo is stored. If cancelled, nothing is stored.

4.4.3 Functional Requirements

- REQ-4.1: Accept JPG, PNG, and PDF inputs up to 10 MB for OCR. [Source: Interview (AI stakeholder)]
- REQ-4.2: Run OCR and return extracted text within 15 seconds at the 90th percentile for a single 300-DPI image on a typical laptop or phone (NFR-2). [Source: Interview (AI stakeholder)]
- REQ-4.3: Display the OCR text in an editable field with the source image alongside it. [Source: Interview (AI stakeholder)]
- REQ-4.4: Save the corrected OCR result as a private recipe owned by the user. [Source: Interview (AI stakeholder)]
- REQ-4.5: Reject files that are not JPG, PNG, or PDF, or are larger than 10 MB, before OCR runs, with an error stating the allowed types and size. [Source: Activity diagram]
- REQ-4.6: Split the extracted text into title, ingredients, and steps fields and show it as a recipe preview for the user to correct. [Source: Activity diagram]
- REQ-4.7: If OCR fails or takes longer than 30 seconds, show an error and let the user retry or switch to manual entry with the image kept for reference. [Source: Team]
- REQ-4.8: Apply the same required-field check as REQ-3.3 before saving a scanned recipe. [Source: Activity diagram]
- REQ-4.9: Run OCR in the user's browser with Tesseract.js, turning up to 5 PDF pages into images with PDF.js first, and never upload the scan to Forkful's servers. [Source: Team]
- REQ-4.10: Discard the scan from the browser once the recipe is saved or the scan is cancelled. [Source: Team]

- REQ-4.11: Check the 100-recipe limit before starting a scan; if it is reached, show "Your Library is full (100 of 100 recipes)" and suggest deleting recipes. [Source: Team]
- REQ-4.12: Let the user add an optional dish photo before saving, compressed as in REQ-3.7; the scan itself is never kept as the photo. [Source: Team]
- REQ-4.13: Split each ingredient line from OCR into quantity (including fractions such as 1½ and ranges such as 2-3), unit, name, and note, and show the split in the preview for the user to correct. Lines that cannot be split are kept as plain text and are not scaled of converted. [Source: Team]

4.5 Delete Recipes

4.5.1 Description and Priority

- Lets a registered user remove any saved copy or private recipe they own.
- Priority: Medium

4.5.2 Stimulus/Response Sequences

1. User selects a recipe and chooses "Delete."
2. System asks for confirmation.
3. On confirm, the system permanently deletes the user-owned record; catalog original is unaffected.

Table 4.5.1 UC-5 Tabular Specification

Use Case	UC-5: Delete a Recipe
Actor	Home Cook (registered user)
Precondition	The Home Cook is logged in and owns at least one saved copy or private recipe.
Main Flow	1. The Home Cook selects one or more recipes in the Library and chooses "Delete." 2. The system asks for confirmation, stating how many recipes will be deleted and that this cannot be undone. 3. The Home Cook confirms. 4. The system permanently deletes the recipes and their photos, removes them from the Library, and confirms.
Alternative Flows	A1 (cancel, step 3): Nothing is changed. A2 (recipe already deleted, step 4): The system says the recipe was already deleted and refreshes the Library.
Postcondition	The selected recipes and their photos are gone from the Library, and catalog originals are unaffected.

4.5.3 Functional Requirements

- REQ-5.1: Allow a user to delete any saved copy or private recipe they own. [Source: Team]
- REQ-5.2: Require explicit confirmation before deleting a recipe. [Source: Team]
- REQ-5.3: Deleting a saved copy shall not affect the catalog original. [Source: Team]
- REQ-5.4: Allow the user to select several recipes and delete them with one confirmation. [Source: Team]

- REQ-5.5: Permanently delete recipes and their photos after the confirmation in REQ-5.2. [Source: Team]

4.6 Admin Account Management

4.6.1 Description and Priority

- Gives System Admin a console to view, disable, re-enable, and delete user accounts, change roles, and watch database storage.
- Priority: Medium

4.6.2 Stimulus/Response Sequences

1. System Admin opens admin console → system displays user accounts with role, status, and recipe count, and the database storage used.
2. System Admin disables, re-enables, or deletes an account, or changes its role → system applies the change and shows the new status.

Table 4.6.1 UC-6 Tabular Specification

Use Case	UC-6: Manage Parser and OCR Jobs
Actor	System Administrator
Precondition	The System Admin is logged in with the admin role.
Main Flow	1. The System Admin opens the admin console. 2. The system lists accounts with username, email, role, status, and recipe count, and shows the database storage used. 3. The System Admin selects an account and chooses disable, re-enable, delete, or change role. 4. The system asks for confirmation for disable and delete. 5. The system applies the change and shows the new status.
Alternative Flows	A1 (admin selects their own account, step 3): The system does not allow disabling, deleting, or demoting it. A2 (cancel at confirmation, step 4): Nothing is changed. A3 (storage above 80% of the free limit, step 2): The system shows a warning on the console (NFR-8).
Postcondition	The account is in the requested state.

4.6.3 Functional Requirements

- REQ-6.1: Provide System Admin with a console listing every account's username, email, role, status, and recipe count. [Source: Interview (AI stakeholder)]
- REQ-6.2: Allow System Admin to disable and re-enable an account; a disabled account cannot log in, and its recipes are kept. [Source: Team]
- REQ-6.3: Allow System Admin to delete an account after confirmation, removing its data as in REQ-7.5. [Source: Team]
- REQ-6.4: Show System Admin the database and photo storage used against the free limits (0.5 GB and 1 GB), and warn at 80% of either (NFR-8). [Source: Team]

4.7 User Profiles

4.7.1 Description and Priority

- Lets a registered user view and update their profile and manage their account, including deleting all attached data (*§2.4 Privacy constraint)*
- Priority: Medium

4.7.2 Stimulus/Response Sequences

1. The user selects “Profile,” and the system shows their username and email.
2. The user edits a field and saves, and the system validates, saves and confirms.
3. The user selects “Delete Account” and confirms, and the system deletes the account and its data and ends the session.

Table 4.7.1 UC-7 Tabular Specification

Use Case	UC-7: Manage Profile and Delete Account
Actor	Home Cook (registered user)
Precondition	The Home Cook is logged in.
Main Flow	1. The Home Cook selects “Profile.” 2. The system shows the username and email. 3. The Home Cook changes the username, email, or password and enters their current password. 4. The system validates and saves the change; for a new email, it sends a verification link and keeps the old email until the link is used. 5. The system confirms the change.
Alternative Flows	A1 (wrong current password, step 4): The system rejects the change and nothing is updated. A2 (delete account, step 3): The Home Cook selects “Delete Account” and confirms; the system permanently deletes the account and all its recipes, ends the session, and shows a confirmation. A3 (new email not verified within 24 hours): The pending email change is discarded.
Postcondition	The profile shows the saved changes, or the account and all its data are deleted.

4.7.3 Functional Requirements

- REQ-7.1: Display the user’s username and email on the Profile page. [Source: Interview (AI stakeholder)]
- REQ-7.2: Allow the user to change their username and password. [Source: Interview (AI stakeholder)]
- REQ-7.3: Send verification link to a new email address, and replace the current one after the link is used. [Source: Interview (AI stakeholder)]
- REQ-7.4: Require the current password before changing the email or password. [Source: Interview (AI stakeholder)]
- REQ-7.5: Allow a user to delete their own account after explicit confirmation, permanently deleting the account and every saved and private recipe in their library once confirmed. [Source: Interview (AI stakeholder)]

- REQ-7.6: Remove a deleted account's data from the live database immediately and from backups within 7 days. [Source: Team]
- REQ-7.7: Discard a pending email change if the new address is not verified within 24 hours. [Source: Team]
- REQ-7.8: Allow users to choose US units, metric units, or the original units of each recipe as a profile preference; the default is the original units. [Source: Team]

4.8 Account Creation

4.8.1 Description and Priority

- Lets guests create an account with a user, email, and password so they can save, create, scan, and delete recipes.
- Priority: High

4.8.2 Stimulus/Response Sequences

1. The guest selects "Create Account," and the system shows the registration form.
2. The guest submits the form, and the system validates the entries, creates an unverified account and emails a verification link.
3. The guest opens the link, and the system verifies the email, activates the account and prompts login.

Table 4.8.1 UC-8 Tabular Specification

Use Case	UC-8: Create an Account
Actor	Primary: Casual Browser (guest) Supporting: Email service
Precondition	The guest is not logged in and has an email address.
Main Flow	1. The guest selects "Create Account." 2. The system shows a form for username, email, password, and age confirmation. 3. The guest fills in the form and submits it. 4. The system checks that the username and email are unused, the password has at least 8 characters, and the guest confirmed they are at least 13. 5. The system creates an unverified account and emails a verification link that is valid for 24 hours. 6. The guest opens the link. 7. The system activates the account and prompts the guest to log in.
Alternative Flows	A1 (username or email in use, step 4): The system names the field in an error message and keeps the other entries. A2 (age not confirmed, step 4): The system does not create the account and explains the age requirement (COPPA). A3 (password too short, step 4): The system states the password rule and returns to step 3. A4 (link expired, step 6): The system says the link expired and offers to send a new one.
Postcondition	An active, verified account exists, and the user can log in.

4.8.3 Functional Requirements

- REQ-8.1: Allow a guest to create an account with a username, email, and password. [Source: Interview (AI stakeholder)]
- REQ-8.2: Reject a username or email already in use, with an error message stating such. [Source: Interview (AI stakeholder)]
- REQ-8.3: Require the guest to confirm they are at least 13 years old. [Source: Interview (AI stakeholder)]
- REQ-8.4: Send a verification link when the account is created. [Source: Interview (AI stakeholder)]
- REQ-8.5: Require passwords of at least 8 characters. [Source: Interview (AI stakeholder)]
- REQ-8.6: Expire the verification link after 24 hours and allow the user to request a new one. [Source: Interview (AI stakeholder)]
- REQ-8.7: Block login until the email address is verified. [Source: Interview (AI stakeholder)]
- REQ-8.8: Delete accounts that are never verified after 7 days. [Source: Team]

4.9 Login and Authentication

4.9.1 Description and Priority

- Lets registered users log in, log out, and reset a forgotten password.
- Priority: High

4.9.2 Stimulus/Response Sequences

1. User selects “Log In” → system shows the login form.
2. User enters credentials → system starts a session and opens the Library.
3. User selects “Log Out” → system ends the session and returns to Home.

Table 4.9.1 UC-9 Tabular Specification

Use Case	UC-9: Log In and Reset Password
Actor	Home Cook (registered user); also System Admin
Precondition	A verified account exists.
Main Flow	1. The user selects “Log In.” 2. The user enters their username or email and password. 3. The system checks the credentials. 4. The system starts a session and opens the Library (or the admin console for the admin role).
Alternative Flows	A1 (wrong credentials, step 3): The system shows “Email or password is incorrect” and counts a failed attempt. A2 (email not verified, step 3): The system explains this and offers to resend the verification link. A3 (forgot password, step 2): The user selects “Forgot password” and enters their email; the system emails a single-use reset link valid for 1 hour; after the new password is set, the system ends other sessions and returns to step 1.
Postcondition	The user is logged in with their role’s permissions, or remains logged out.

4.9.3 Functional Requirements

- REQ-9.1: Allow a verified user to log in with their username or email and password. [Source: Data flow diagram]
- REQ-9.2: Show one generic message ("Email or password is incorrect") for any failed login. [Source: Team]
- REQ-9.3: Provide a Log Out button on every page that ends the session. [Source: Team]
- REQ-9.4: End a session after 30 minutes of inactivity (NFR-15). [Source: Team]
- REQ-9.5: Send a single-use password reset link that expires after 1 hour, and end all other sessions once the password is reset. [Source: Team]
- REQ-9.6: Show admin tools only to accounts with the admin role. [Source: Interview (AI stakeholder)]

4.10 Library View and Organization

4.10.1 Description and Priority

- Lets a registered user view, search, and organize their saved copies and private recipes with tags.
- Priority: High

4.10.2 Stimulus/Response Sequences

1. User opens the Library → system lists their recipes.
2. User enters a search or filter → system shows matching recipes.
3. User adds or removes a tag → system saves it and updates the list.

Table 4.8.1 UC-10 Tabular Specification

Use Case	UC-10: Organize the Library
Actor	Home Cook (registered user)
Precondition	The Home Cook is logged in.
Main Flow	1. The Home Cook opens the Library. 2. The system lists their saved copies and private recipes, labeled by type, with the Modified indicator where it applies. 3. The Home Cook searches, sorts, or filters by tag. 4. The Home Cook adds or removes tags on a recipe. 5. The system saves the change and updates the list.
Alternative Flows	A1 (tag name already exists, step 4): The system reuses the existing tag. A2 (empty Library, step 2): The system shows an empty-Library message with links to the catalog and "New Private Recipe."
Postcondition	The organization changes are saved and visible only to the Home Cook.

4.10.3 Functional Requirements

- REQ-10.1: List every saved copy and private recipe the user owns, labeled "Saved copy" or "Private," with the Modified indicator where it applies. [Source: Data flow diagram]

- REQ-10.2: Allow searching the Library by title, ingredient, and tag. [Source: Team]
- REQ-10.3: Allow sorting the Library by title, date added, and last modified. [Source: Team]
- REQ-10.4: Allow adding and removing user-defined tags on any recipe in the Library. [Source: §2.2]
- REQ-10.5: Show an empty-Library message with links to the catalog and "New Private Recipe." [Source: Team]
- REQ-10.6: Show the user how many of their 100 recipes they are using, and warn them when they reach 90. [Source: Team]
- REQ-10.7: Show each recipe's thumbnail in the Library list; a recipe without a photo shows a placeholder image. [Source: Team]

4.11 Cooking Mode

4.11.1 Description and Priority

- Gives any user a step-by-step recipe view for cooking that keeps the device screen awake (§3.1).
- Priority: Medium

4.11.2 Stimulus/Response Sequences

1. User selects "Cooking Mode" on a recipe → system shows the step view and keeps the screen awake.
2. User selects Next or Previous → system shows that step.
3. User finishes or exits → system returns to Browser mode.

Table 4.11.1 UC-11 Tabular Specification

Use Case	UC-11: Use Cooking Mode
Actor	Home Cook (registered user) or Casual Browser (guest)
Precondition	A recipe detail page is open.
Main Flow	1. The user selects "Cooking Mode." 2. The system switches to the Cooking mode UI, shows the first step in large text with the ingredient list available, and keeps the screen awake. 3. The user moves through the steps with Next and Previous. 4. The user finishes the last step. 5. The system releases the screen lock and returns to Browser mode.
Alternative Flows	A1 (screen lock not supported, step 2): The system continues in Cooking mode and tells the user the screen may sleep. A2 (tab switched or 60 minutes passed): The system releases the screen lock and returns to Browser mode. A3 (user exits early, step 3): Same as step 5.
Postcondition	Browser mode is restored, and the recipe is unchanged.

4.11.3 Functional Requirements

- REQ-11.1: Show a "Cooking Mode" button on every recipe detail page. [Source: §3.1]
- REQ-11.2: Display one step at a time in large text with Next and Previous buttons and an ingredient list the user can open. [Source: Team]

- REQ-11.3: Keep the device screen awake while Cooking mode is active. [Source: §3.1]
- REQ-11.4: Exit Cooking mode and release the screen lock when the last step is finished, the user exits, the browser tab is switched, or 60 minutes pass. [Source: §3.1]
- REQ-11.5: If the browser cannot keep the screen awake, stay in Cooking mode and tell the user the screen may sleep. [Source: Team]
- REQ-11.6: Allow changing the number of servings to scale ingredient amounts on screen without changing the stored recipe. Priority: Low. [Source: Team]
- REQ-11.7: Provide a print-friendly view of any recipe. Priority: Low. [Source: Team]

4.12 Catalog Curation

4.12.1 Description and Priority

- Gives the System Admin tools to find failed imports and correct or remove bad catalog entries.
- Priority: Medium

4.12.2 Stimulus/Response Sequences

1. System Admin opens the curation dashboard → system lists failed and flagged entries.
2. System Admin edits or removes an entry → system updates the catalog.

Table 4.12.1 UC-11 Tabular Specification

Use Case	UC-11: Review and Correct Catalog Entries
Actor	System Administrator
Precondition	The System Admin is logged in.
Main Flow	1. The System Admin opens the curation dashboard. 2. The system lists failed parses and flagged catalog entries with their source URL and problem. 3. The System Admin selects an entry. 4. The System Admin edits its fields or removes it. 5. The system saves the change and updates the catalog.
Alternative Flows	A1 (remove an entry users have saved, step 4): The system warns that saved copies will stay in users' libraries (BR-6) and proceeds on confirmation. A2 (another admin changed the entry meanwhile, step 5): The system shows the latest version and asks the System Admin to reapply the change.
Postcondition	The catalog is corrected, and saved copies are unchanged.

4.12.3 Functional Requirements

- REQ-12.1: Provide a curation dashboard listing failed parses and flagged catalog entries with source URL and error. [Source: §2.3]
- REQ-12.2: Allow the System Admin to edit any field of a catalog recipe. [Source: §2.3]
- REQ-12.3: Allow the System Admin to remove a catalog recipe; saved copies of it stay in users' libraries (BR-6). [Source: §2.3]

4.13 Parse a Recipe from a URL

4.13.1 Description and Priority

- Lets the System Admin add a single recipe to the catalog by submitting its web address. In v1.0 this is the only way the catalog grows (§6.1).
- Priority: Low

4.13.2 Stimulus/Response Sequences

1. System Admin enters a URL → system checks it and parses the page.
2. System Admin reviews the preview and confirms → system adds the recipe to the catalog.

Table 4.13.1 UC-13 Tabular Specification

Use Case	UC-13: Parse a Recipe from a URL
Actor	Primary: System Administrator Supporting: Recipe import function
Precondition	The System Admin is logged in, and the parser is not paused.
Main Flow	1. The System Admin opens "Parse URL" in the admin console. 2. The System Admin enters a recipe page URL and submits it. 3. The system checks the URL format, the site's terms and robots.txt rules, and whether the URL is already in the catalog. 4. The recipe import function fetches the page and reads its schema.org Recipe data: title, ingredients, steps, and source. 5. The system shows the parsed recipe as a preview. 6. The System Admin confirms. 7. The system adds the recipe to the catalog.
Alternative Flows	A1 (invalid URL or not a recipe page, step 3 or 4): The system shows the reason; the catalog is unchanged. A2 (site disallows parsing, step 3): The system rejects the URL and cites the site's rules (BR-3). A3 (already in catalog, step 3): The system shows the existing catalog entry. A4 (admin rejects the preview, step 6): The system discards the parsed recipe.
Postcondition	A new catalog recipe is searchable, or the catalog is unchanged.

4.13.3 Functional Requirements

- REQ-13.1: Allow System Admin to submit a recipe page URL for import. [Source: Data flow diagram]
- REQ-13.2: Reject URLs that are malformed, are not recipe pages, or are on sites whose terms or robots.txt disallow parsing, and show the reason. [Source: BR-3]
- REQ-13.3: If the URL is already in the catalog, show the existing entry instead of adding a duplicate. [Source: Team]
- REQ-13.4: Show the parsed recipe as a preview and add it to the catalog only after the admin confirms. [Source: Team]

- REQ-13.5: Make a newly added recipe appear in catalog search as soon as it is added. [Source: Team]
- REQ-13.6: Read the recipe from the page's schema.org Recipe (JSON-LD) markup; if the page has none, reject it as not a recipe page. [Source: Team]
- REQ-13.7: Do not copy photos from the source page, since they are usually copyrighted; an imported recipe shows a placeholder until the System Admin adds a photo the team owns or that is freely licensed (BR-12). [Source: Team]
- REQ-13.8: Split imported ingredient lines into quantity, unit, name, and note as in REQ-4.13. [Source: Team]

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement	Source
NFR-1 (Performance)	Return catalog searches within four seconds with a catalog of up to 10,000 recipes while serving the users in NFR-6.	Interview (AI stakeholder)
NFR-2 (Performance)	Return OCR text within 15 seconds at the 90th percentile for a single 300-DPI image on a typical laptop or phone, since OCR runs in the browser. (Accuracy is NFR-20.)	Interview (AI stakeholder)
NFR-3 (Performance)	Load any page within 3 seconds at the 95th percentile on a 5 Mbps connection; lists load thumbnails (at most 20 KB each), not full photos.	Team
NFR-4 (Performance)	Confirm a recipe save within 5 seconds of the user clicking Save.	Team (moved from §2.4)
NFR-5 (Performance)	Load the Library within 2 seconds for a full library of 100 recipes (BR-10).	Team
NFR-6 (Capacity)	Support 100 concurrent users while still meeting NFR-1 and NFR-3.	Team
NFR-7 (Scalability)	Meet NFR-1 with a catalog of up to 20,000 recipes.	Team

NFR-8 (Capacity)	Keep the database under 400 MB (80% of Neon's free 0.5 GB) and recipe photos under 800 MB (80% of Vercel Blob's free 1 GB). At about 5 KB of text and 120 KB of photos per recipe, a 500-recipe catalog plus about 6,000 user recipes with photos fits.	Team & Interview (AI stakeholder)

5.2 Safety Requirements

ID	Requirement	Source
NFR-9 (Safety)	Cache recipe drafts in the browser as the user types and restore them after an accidental close or crash	Team (moved from §2.4)
NFR-10 (Safety)	Warn the user before leaving a page with unsaved changes.	Team
NFR-11 (Safety)	Show a notice on every recipe that ingredient and allergen information comes from the source or the user and is not verified by Forkful.	Team

5.3 Security Requirements

ID	Requirement	Source
NFR-12 (Security)	Encrypt all data in transit with TLS 1.2 or newer, keep the database encrypted at rest, and encrypt backup files before they are stored	Interview (AI stakeholder)
NFR-13 (Security)	Enforce role-based access control over recipes, where users can read/write only their own saved copies and private recipes.	Interview (AI stakeholder)
NFR-14 (Security)	Store passwords only as salted hashes made with Argon2id or bcrypt; never store or log them in plain text.	Interview (AI stakeholder)

NFR-15 (Security)	End user sessions after 30 minutes of inactivity.	Team
NFR-16 (Security)	Check scans and dish photos by their content, not only their extension, in the browser; scans are never uploaded, and only compressed WebP photos are stored.	Team
NFR-17 (Security)	Never return private recipes or saved copies from catalog search or any public page; verified by automated tests with 0 leaks.	Team
NFR-18 (Security)	The Catalog Service connects with a read-only database role; only the recipe import and curator functions can write the catalog schema.	Team

5.4 Software Quality Attributes

ID	Requirement	Source
NFR-19 (Usability)	Site will provide feedback on whether actions had or did not have an effect	Ethnography (AI stakeholder)
NFR-20 (Accuracy)	OCR reaches at least 90% character accuracy on a test set of printed recipes scanned at 300 DPI.	Interview (AI stakeholder)
NFR-21 (Usability)	A new user can save a copy of a catalog recipe in 3 clicks or fewer from the search results.	Team
NFR-22 (Usability)	Meet WCAG 2.1 Level AA accessibility.	Team
NFR-23 (Usability)	Display correctly without horizontal scrolling on screens 320 px wide and larger,	Team
NFR-24 (Usability)	Every error message states the problem, its cause, and a suggested action (§3.1).	Team

NFR-25 (Reliability)	Be available 99% of each month, excluding maintenance announced 24 hours ahead. This is a target: the free plans give no uptime guarantee.	Team
NFR-26 (Reliability)	Recovery point objective of 24 hours and recovery time objective of 4 hours (§3.5).	Team (moved from §3.5)
NFR-27 (Reliability)	Accounts, the Library, and OCR keep working when a recipe import fails or a source site is down (§2.8).	Team
NFR-28 (Maintainability)	Each architecture component is a separate module behind its own interface, with at least 70% unit-test coverage on service code.	Interview (AI stakeholder)
NFR-29 (Maintainability)	Usage limits (the recipe cap and the storage warning level) can be changed by configuration without redeploying.	Team
NFR-30 (Compatibility)	Support the latest two major versions of Chrome, Safari, and Firefox on desktop and mobile.	Team
NFR-31 (Accuracy)	Scaled and converted amounts are within 1% of the exact value before rounding, checked against a test table of at least 50 conversions.	Team & AI stakeholder

5.5 Business Rules

- BR-1: Public catalog records are immutable from the user's perspective.
- BR-2: Private recipes are visible only to their owner.
- BR-3: Web parsing must respect source-site terms of service.
- BR-4: Users must be at least 13 years old (COPPA).
- BR-5: Each email address can belong to only one account.
- BR-6: A saved copy stays in the user’s Library when its catalog original is changed or removed.
- BR-7: Only System Admins can add, change, or remove catalog recipes.
- BR-8: Every catalog recipe shows the name of and a link to its source site.
- BR-9: A user can read or change only the recipes they own.
- BR-10: Each user can store up to 100 recipes (saved copies plus private recipes).
- BR-11: Forkful is non-commercial, with no ads or payments, as required by Vercel’s free Hobby plan.

- BR-12: Catalog photos must be owned by the team or freely licensed; Forkful does not copy photos from source sites.

6. Other Requirements

6.1 Database and OCR Engine Decisions

These choices resolve the open items in Appendix C. They follow from Forkful's structure: users own recipes, saved copies point back to a read-only catalog, ownership must be enforced on every request (NFR-4), and catalog search must stay under 4 seconds for 50,000 recipes (NFR-1).

Decision	Choice	Why	Rejected alternatives
Database engine	PostgreSQL 18 on Neon's free plan	Relational data (users own recipes, copies reference catalog rows, tags are many-to-many) fits tables and foreign keys. Transactions keep "save a copy" and account deletion all-or-nothing. Schemas and database roles give the catalog a read-only role (NFR-18), and row-level security adds a second check that users only reach their own rows (NFR-13). Built-in backups and point-in-time recovery cover NFR-26. Free and open source. Neon's free plan costs $0 with no card and includes 0.5 GB of storage and a 6-hour instant-restore window.	MongoDB: ownership links, joins, and cascading deletes are harder, and there is no row-level security. MySQL 8: no row-level security and weaker full-text ranking. SQLite: one writer at a time and no roles; fine only for local testing.
Search	PostgreSQL full-text search (tsvector with a GIN index) plus the pg_trgm extension for typo-tolerant title matches	A 10,000 to 20,000-recipe catalog is small enough for the database's own search to meet NFR-1 and NFR-7, with no extra service to keep in sync.	Elasticsearch/OpenSearch: another server to run and sync; revisit only if the catalog grows well past 100,000 recipes.
Schema layout	One Neon database with two schemas: catalog and app	Matches Figure 2-1 and the architecture worksheet. The recipe import and curation functions write catalog; the app reads it through the Catalog Service and writes only app.	Separate database servers or an external catalog API: more cost and setup with no benefit at this size.
Saved-copy storage	Full copy in app.recipe with source_catalog_id and source_hash	Edits never touch the catalog (BR-1), copies survive when the original changes or is removed (BR-6), and "Modified" is found by comparing hashes.	Override records (data flow store D4): every read must merge the original with overrides, which breaks when the

			original changes or disappears.
File storage	Vercel Blob (free 1 GB) for compressed recipe photos only	Photos are compressed in the browser to WebP (about 100 KB plus a 20 KB thumbnail), so about 6,500 recipe photos fit in the free 1 GB. Scans are never stored.	Photos in the database (would fill Neon's 0.5 GB quickly); full-size photos (a few hundred would fill the free limit); Cloudflare R2 (more space, but another account and service).
OCR engine	Tesseract.js in the user's browser, with PDF.js for PDF scans (JPG, PNG, PDF)	Free and open source, needs no server CPU (Vercel's free plan includes only a few hours of active CPU a month), and private recipe images never leave the device.	Tesseract on a server: needs an always-on machine. Cloud OCR (Google Cloud Vision, AWS Textract): costs per page and needs a card
Hosting	Vercel Hobby plan running a Next.js (TypeScript) app	No card, and it cannot bill you: usage pauses at the limit instead. Always on, deploys from GitHub, and runs the API as serverless functions. Personal, non-commercial use only (BR-11).	The other free options compared in §6.5
Email	Resend free plan over its HTTPS API	Sends verification and reset emails free up to 3,000 a month and 100 a day, which covers a class project or small launch	Running a mail server: emails from a new server are often marked as spam, and free hosts block mail ports.
Catalog growth	A seed file of 100 to 500 recipes at launch, plus import by URL (System Admin)	Needs no background server, avoids crawling sites against their terms, and fits Vercel's free plan, where scheduled jobs run at most once a day.	A scheduled crawler on a background worker: needs an always-on server and raises rate-limit and terms-of-service risks.
Backups	Nightly pg_dump by a free scheduled GitHub Actions workflow (last 7 kept), plus Neon's 6-hour instant restore	Meets NFR-26 at $0. Neon's free restore window alone covers only 6 hours.	A paid Neon plan with a longer restore window.
Scheduled cleanup	One daily Vercel cron job	Deletes accounts never verified after 7 days (REQ-8.8); once a day is the most often Vercel's free plan allows.	A separate scheduler service.
Project scope	v1.0 builds the core features;	Keeps the class-project build to one web app, one database, and	Building every feature at once: more services

	ideas for later are listed in Appendix E	one photo store, which all fit free plans.	to host and less time to test the core.

Known limitations: OCR speed depends on the user's device. Tesseract reads printed text well but is weak on handwriting, so handwritten scans will need more correction in the review screen. NFR-20 applies to printed 300-DPI scans.

6.2 Data Requirements

Main data entities (app schema unless noted). Ingredient and step rows exist in both schemas.

Entity	Key attributes	Notes
user	user_id (UUID, PK), username (unique), email (unique), password_hash, role (home_cook, admin), is_verified, unit_preference(us, metric, original), created_at	Deleted with all owned rows on account deletion (REQ-7.5). Recipe count is computed for the 100-recipe limit (BR-10).
recipe	recipe_id (UUID, PK), owner_id (FK users), kind (saved_copy, private), source_catalog_id (FK catalog.recipe, nullable), source_hash, title, description, servings, prep_minutes, cook_minutes, cuisine, photo_path, thumb_path, notes, created_at, updated_at	"Modified" is computed from source_hash.
recipe_ingredient	recipe_id (FK), position, quantity, unit, name, note, raw_text	Ordered list
recipe_step	recipe_id (FK), position, text	Ordererd list
tag, recipe_tag	tag_id, owner_id, name; recipe_id, tag_id	Tag names unique per user
catalog.recipe	catalog_id (PK), title, description, cuisine, tags, source_url (unique), source_site, photo_path, thumb_path, parsed_at, status (active, failed, removed), extra (JSONB), search_vector (tsvector)	Written only by the recipe import and curation functions
auth_token	token_id (UUID, PK), user_id (FK users), purpose (verify, reset, email_change), token_hash, new_email, expires_at, used_at	Single-use links (REQ-7.3, REQ-8.6, REQ-9.5); only a hash of the token is stored
session	session_id (UUID, PK), user_id (FK users), last_seen_at, expires_at	Ends after 30 minutes idle (NFR-15)
catalog_import	import_id (UUID, PK), url, requested_by (FK users), status (pending, imported, failed, rejected), error, catalog_id (FK catalog.recipe, nullable), created_at	Recipe imports by URL; failed ones appear on the curation dashboard (REQ-11.1)

6.3 Data Retention

- Deleted accounts: removed from the live database immediately; only the last 7 nightly backups are kept, so all copies are gone within 7 days.

- Scans (images and PDFs): never uploaded; discarded in the browser after the recipe is saved or cancelled.
- Recipe photos: deleted with their recipe or when replaced (REQ-3.8).
- Unverified accounts: deleted after 7 days.

6.4 Legal and Localization

- Every catalog recipe shows its source site name and link (REQ-1.4, BR-8).
- Requests from source sites to remove their recipes are handled by the System Admin within 5 business days (REQ-11.3).
- Privacy policy and terms of use are linked from every page.
- v1.0 is English only; quantities are shown in the units entered or parsed, or converted to the user's chosen US or metric units (REQ-1.15).

Appendix A: Glossary

OCR	*Optical character recognition. Converts printed or handwritten text into machine-readable, editable digital text.*
Catalog	*The public, read-only collection of recipes parsed from blogs and cooking sites*
Saved copy	*A user's private, editable duplicate of a catalog recipe, linked to its source.*
Private recipe	*A recipe a user entered or scanned; visible only to its owner.*
Library	*A user's private set of saved copies and private recipes.*
Collection	*A user-named group of Library recipes (the "sets" in §2.2).*
Trash	*Holding area for deleted recipes, which can be restored for 30 days. (possible future feature, Appendix E).*
Parser	*The scheduled job that fetches recipe pages and writes them to the catalog.*
Cooking mode	*A step-by-step recipe view that keeps the screen awake (§4.13).*
RBAC	*Role-based access control; permissions depend on the user's role.*
RLS	*Row-level security; a PostgreSQL feature that limits which table rows a user can reach.*
RPO / RTO	*Recovery point objective (most data that can be lost) and recovery time objective (longest time to restore service)*
DPI	*Dots per inch; the resolution of a scanned image.*
Full-text search	*Database search that matches words and ranks results by relevance.*
Tesseract.js	*An open-source OCR library that runs in the web browser.*
Recipe import	*Reading one recipe page by its URL and adding it to the catalog.*
Free tier	*A provider's no-cost plan with fixed usage limits (§6.5).*
WebP	*A compressed image format that keeps photos small for fast pages.*
Thumbnail	*A small version of a recipe photo shown in lists.*
Scaling Unit	*Recalculating ingredient amounts for a different number of servings.*
Unit conversion	*Showing amounts in US or metric units within the same kind of measure (volume, weight, temperature)*

Appendix B: Analysis Models

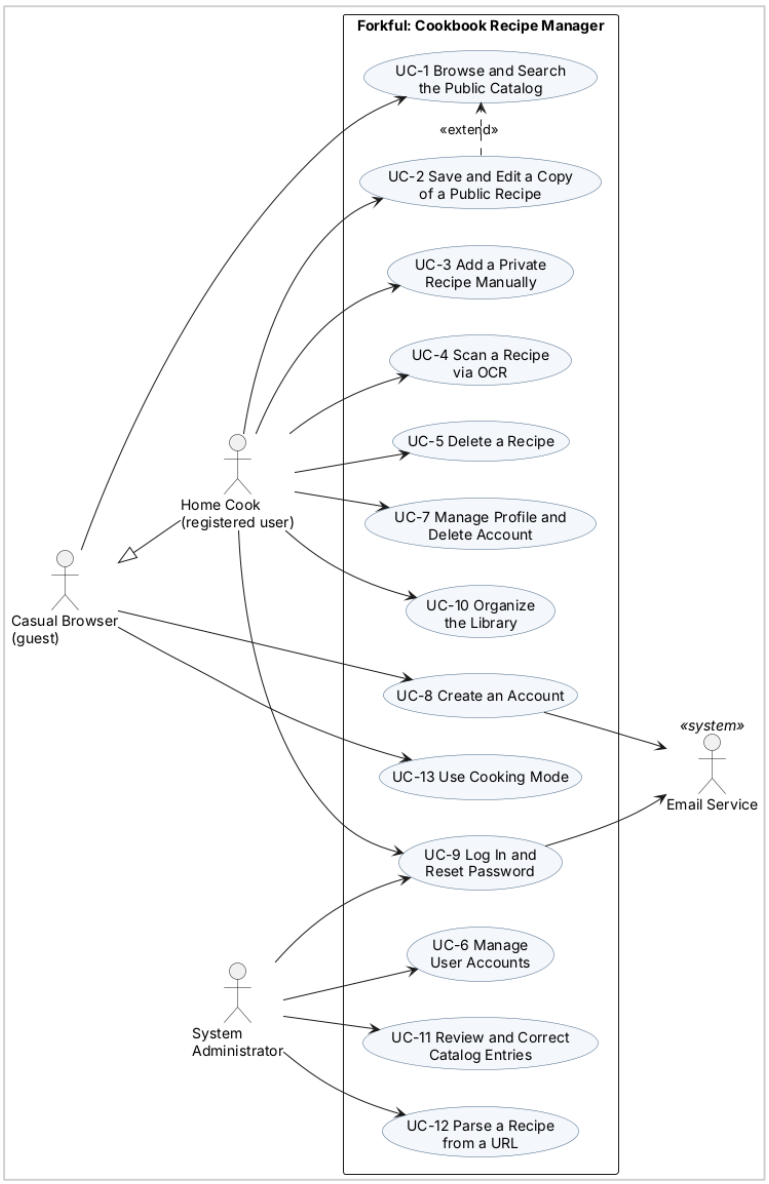


Figure B-1. Use case diagram for Forkful, showing all actors and use cases, including UC-1 and UC-13 (specified in §4)

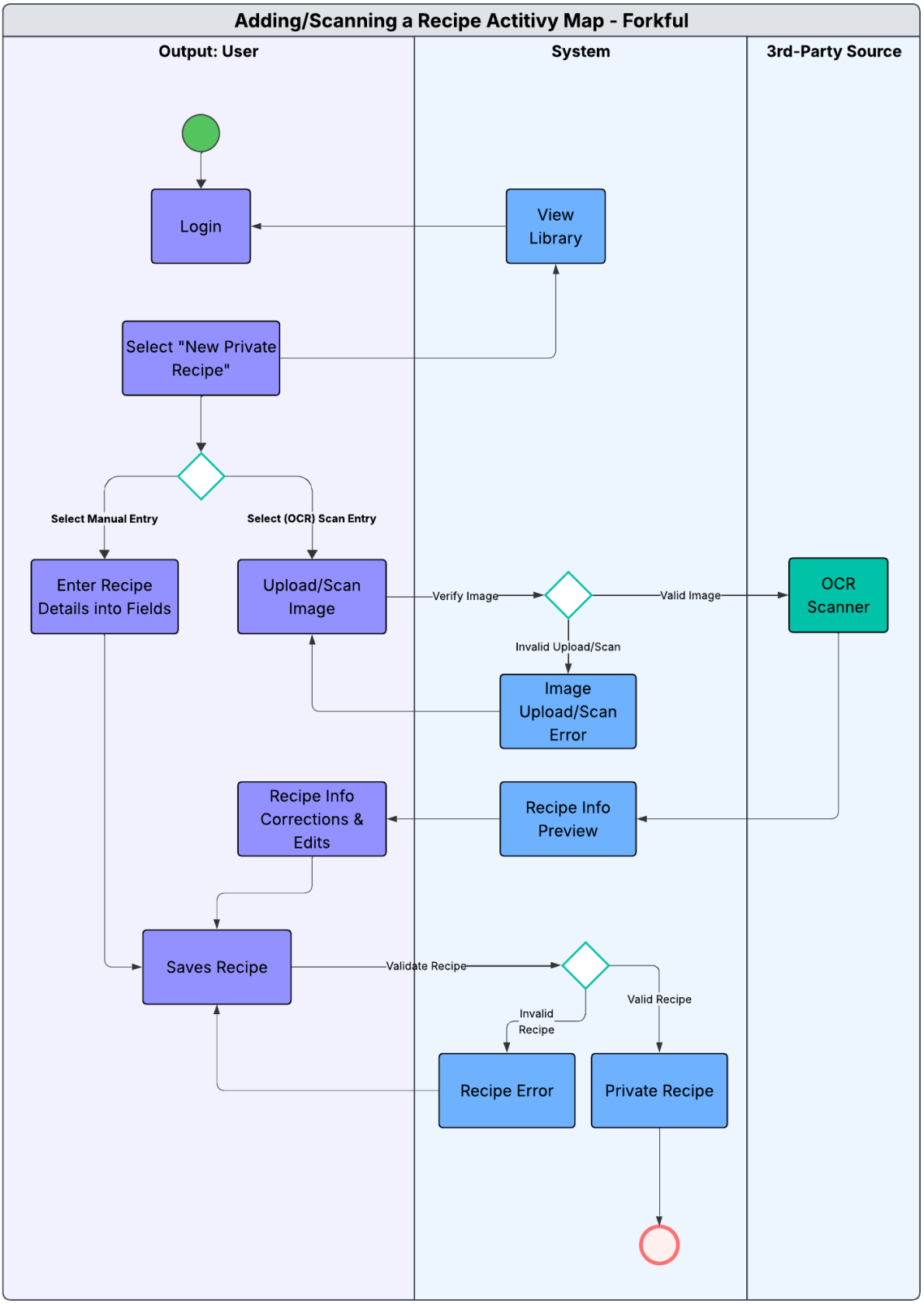


Figure B-2. Activity diagram for adding or scanning a private recipe (UC-3, UC-4).

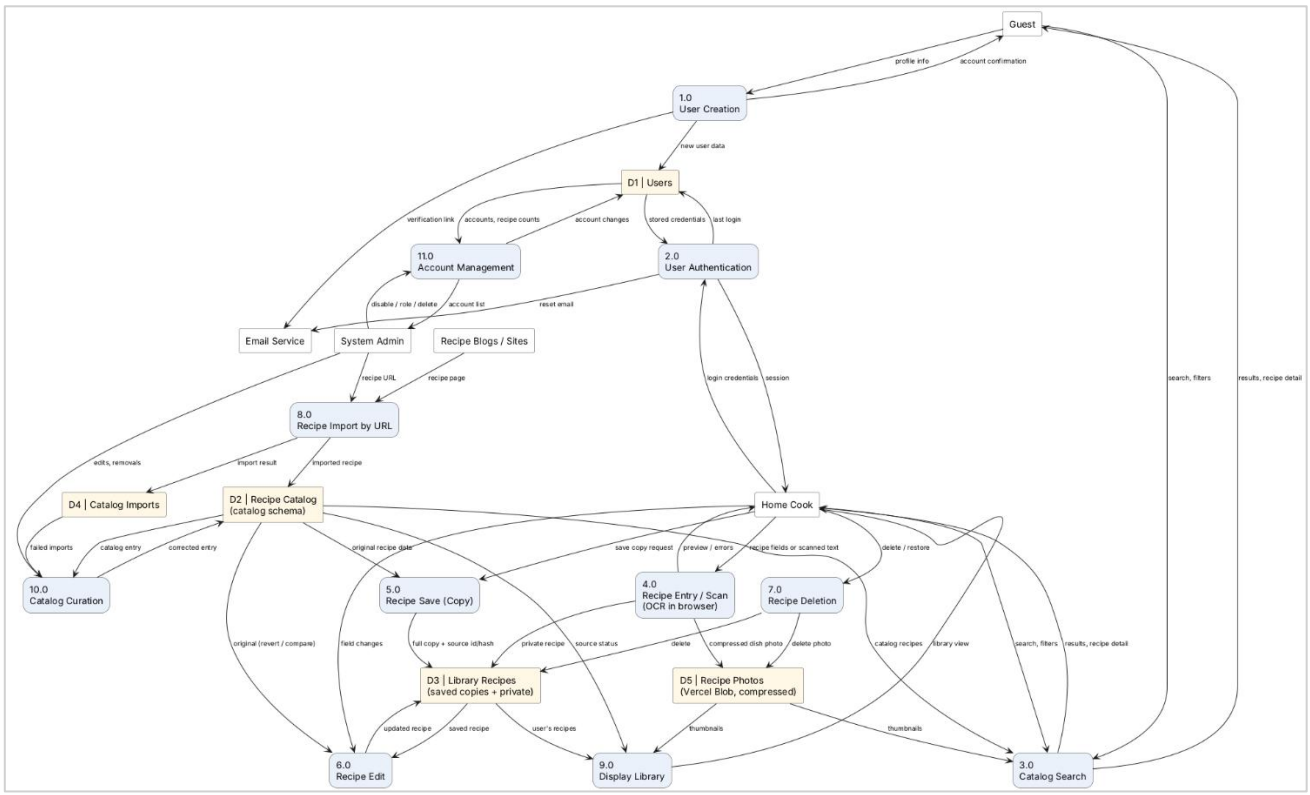


Figure B-3. Level-1 data flow diagram for Forkful. Saved copies and private recipes are stored in D3 and their compressed photos in D5; scans stay in the user's browser (§6.1).

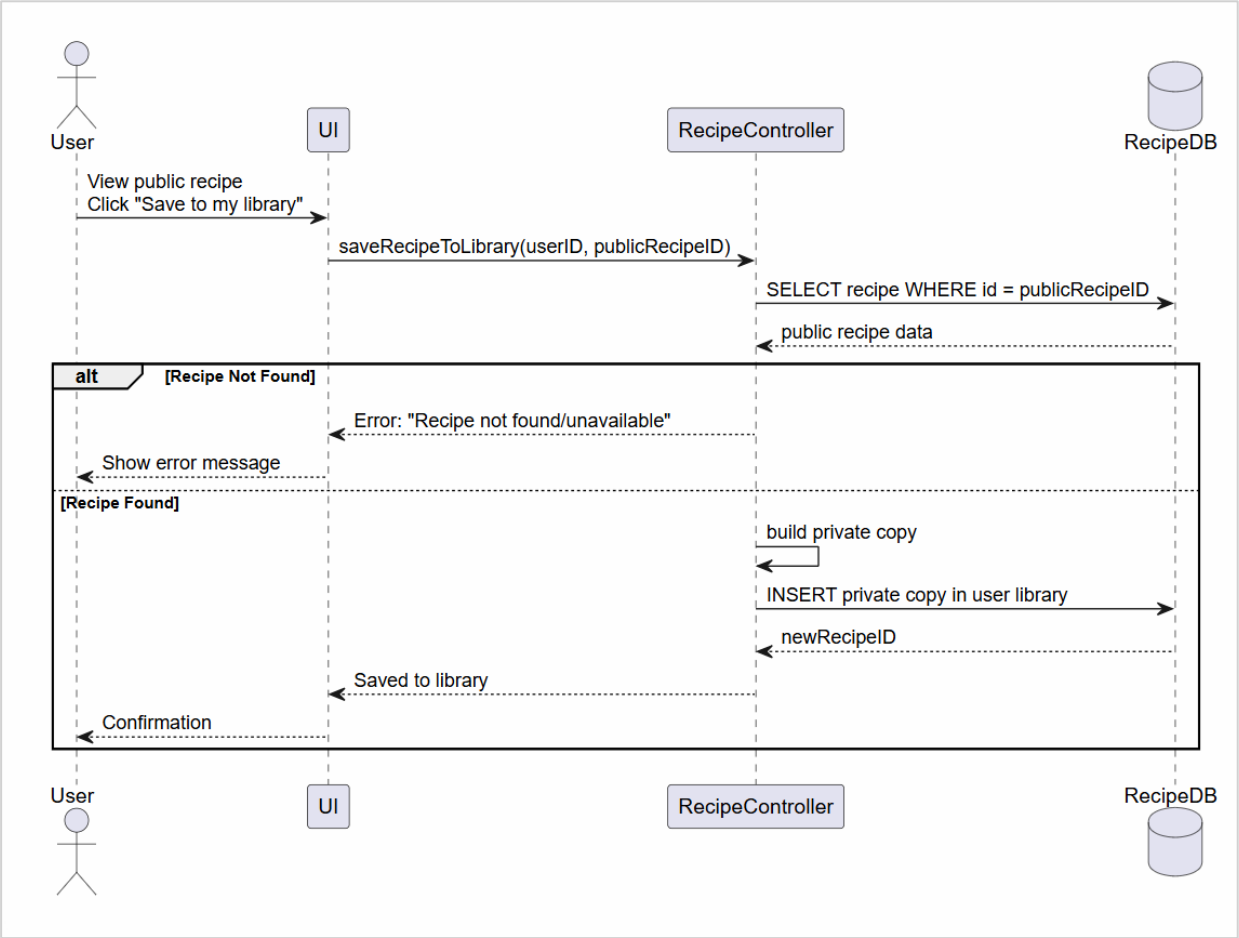


Figure B-4. Sequence diagram for saving a public recipe to the Library (UC-2).

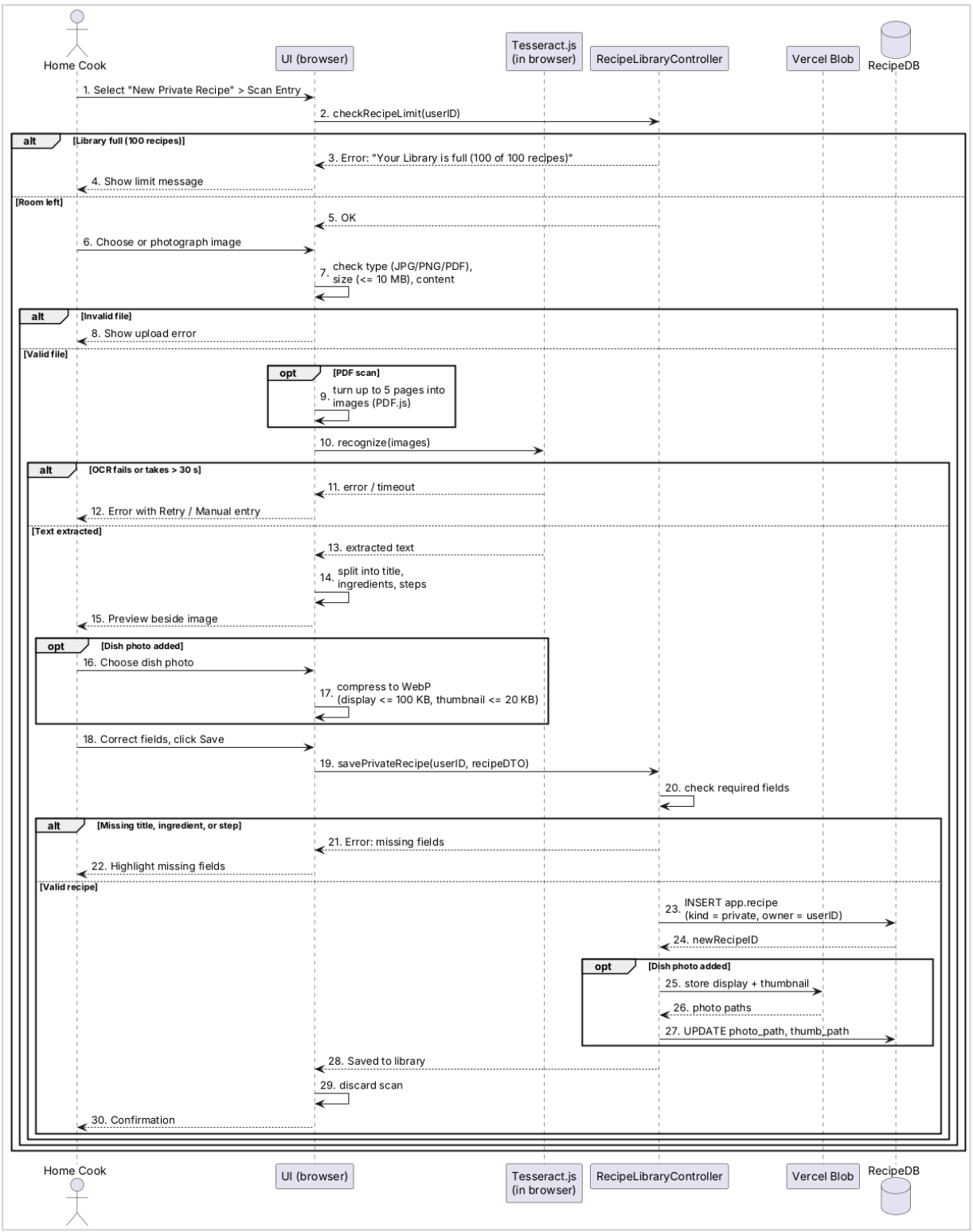


Figure B-5. Sequence diagram for scanning a recipe via OCR in the browser, including PDF scans and an optional dish photo (UC-4).

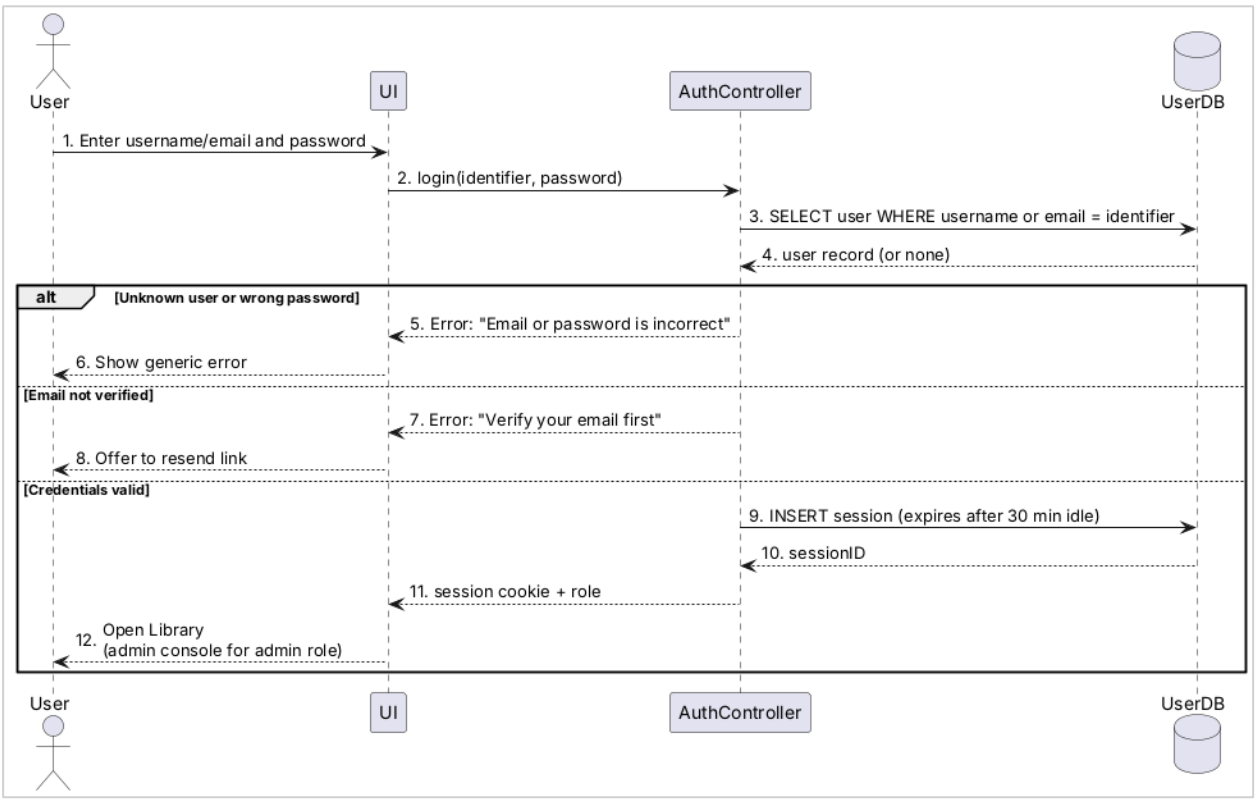


Figure B-6. Sequence diagram for logging in (UC-9)

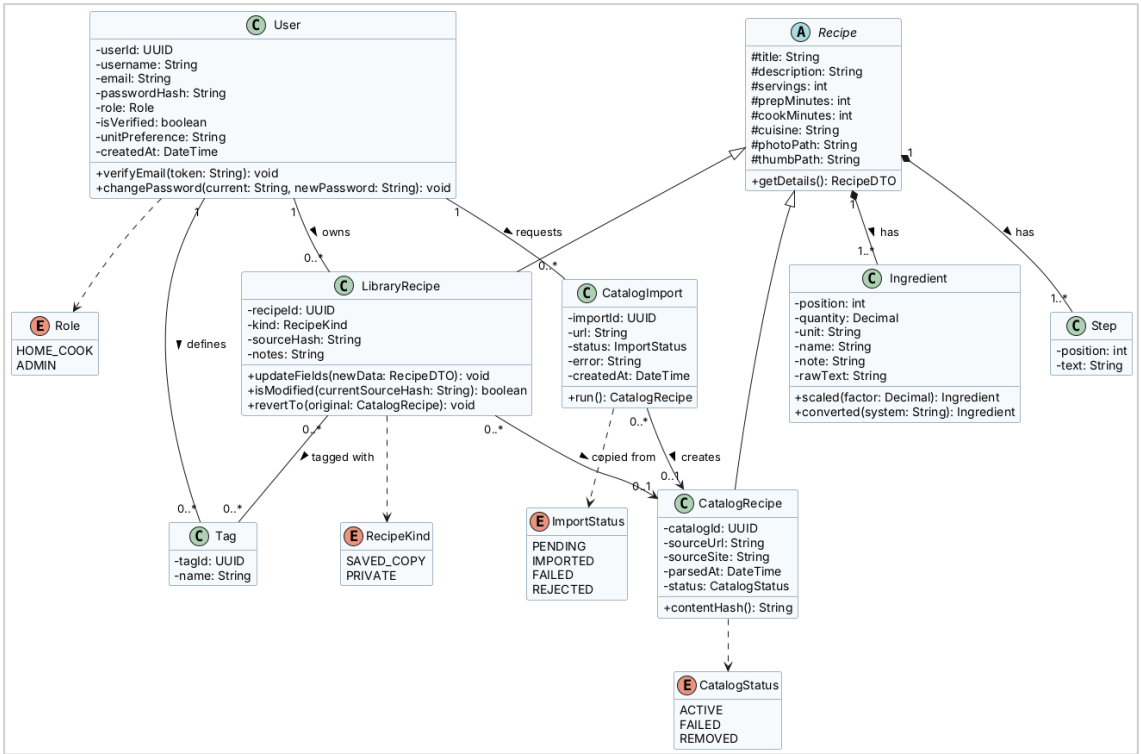


Figure B-7. Domain class diagram: the main entities, their attributes, and how they relate.

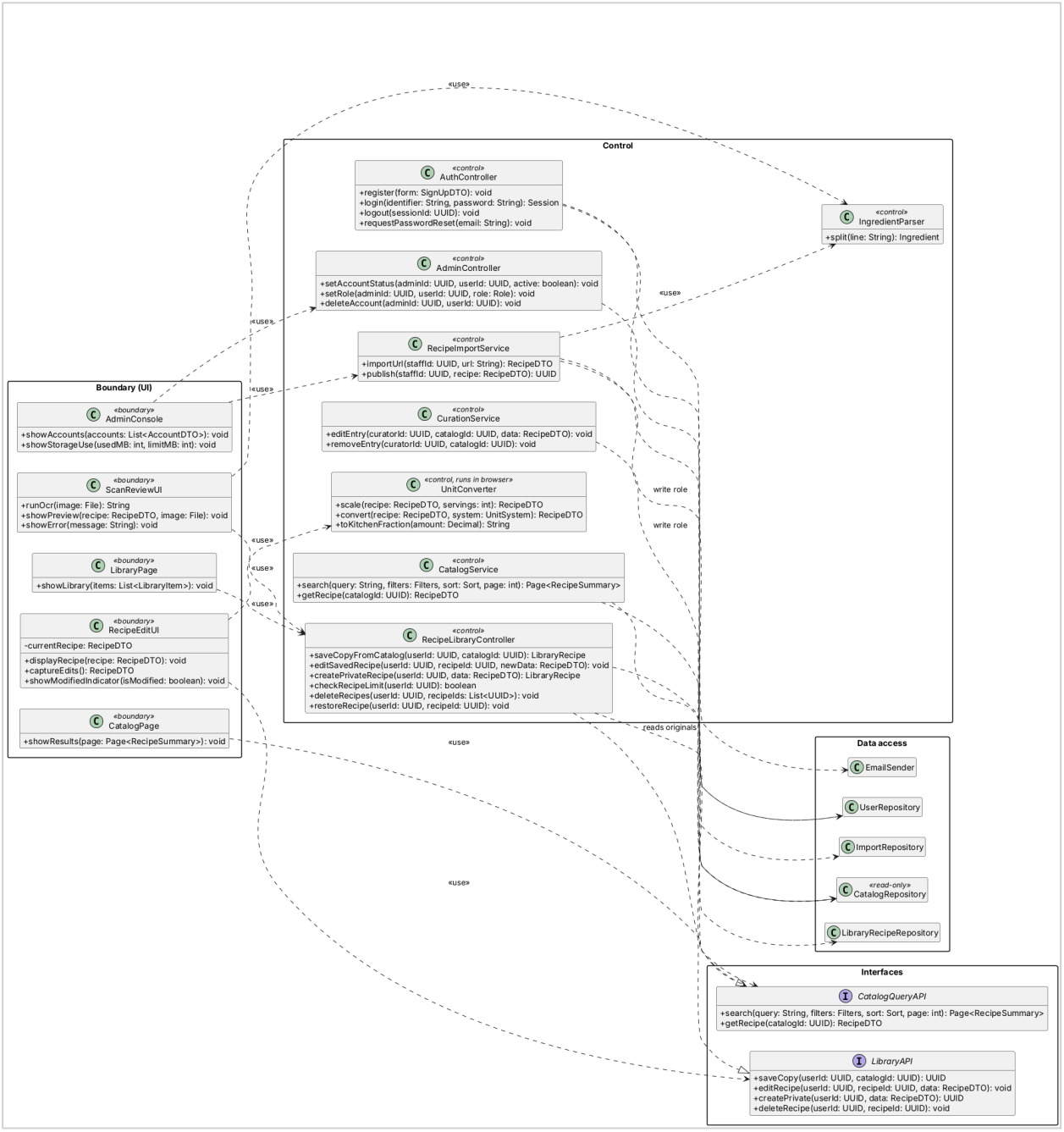


Figure B-8. Design class diagram: boundary, control, and interface classes, named as in the architecture worksheet.

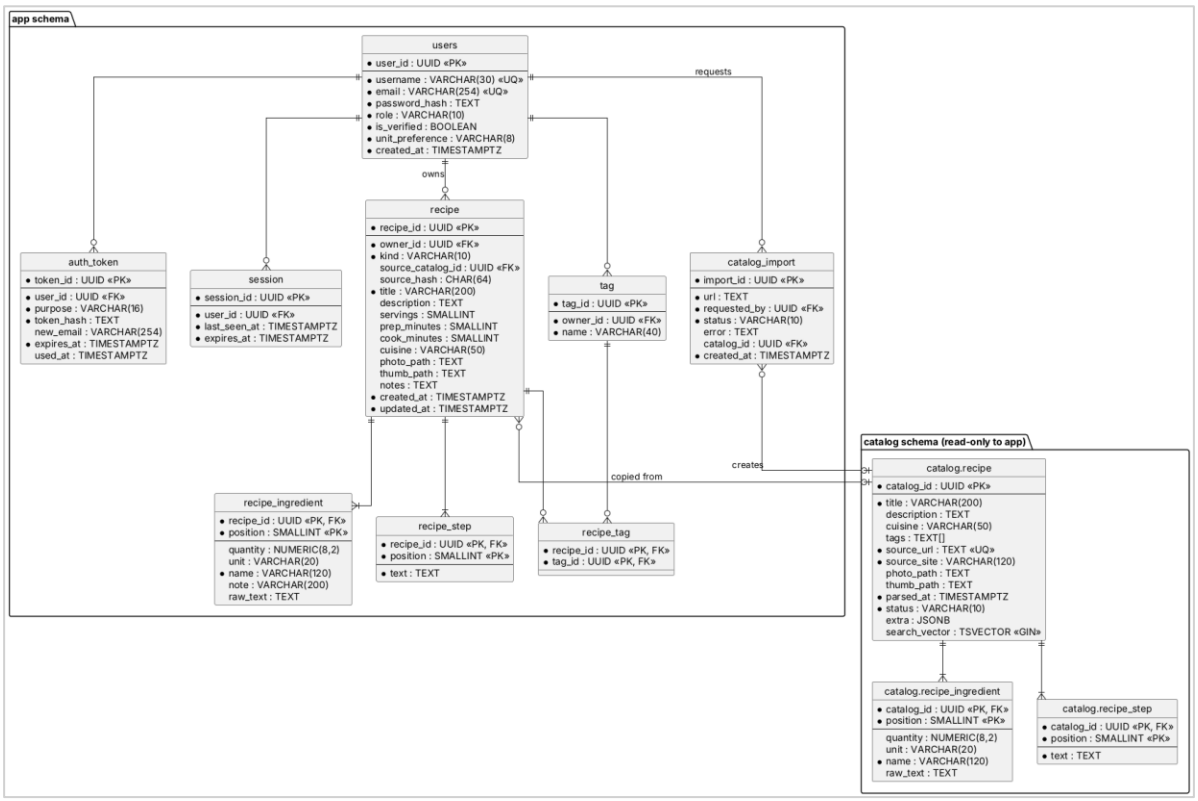


Figure B-9. Entity-relationship diagram for the PostgreSQL 18 database on Neon (§6.1, §6.2).

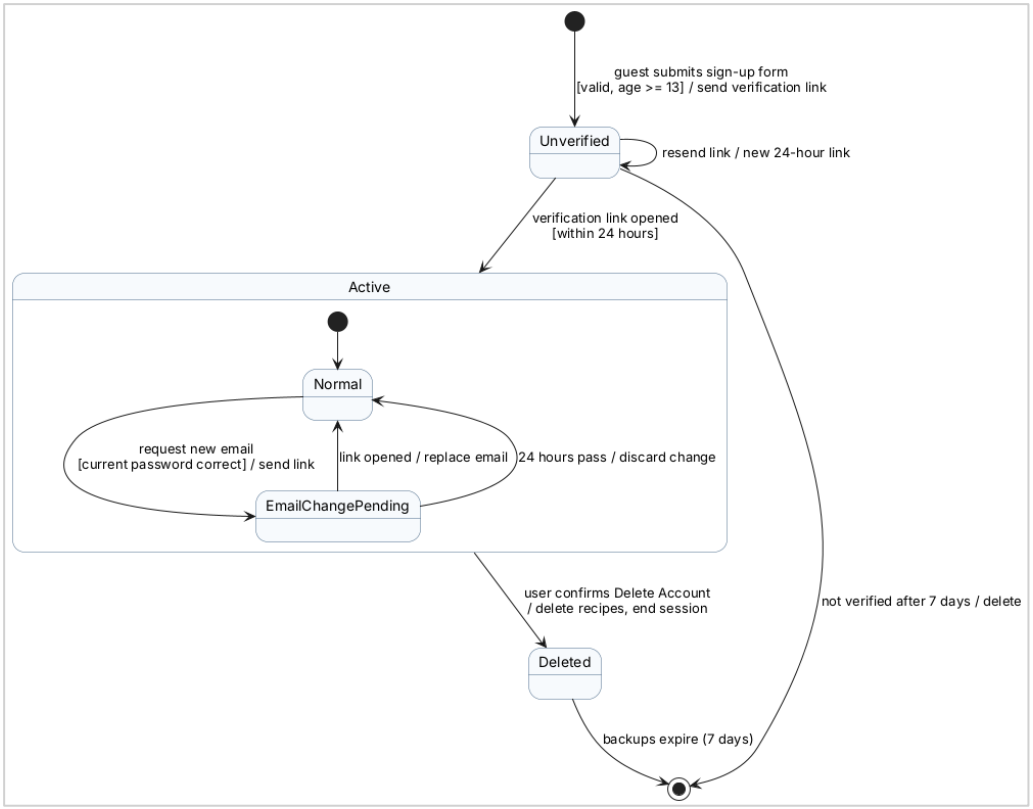


Figure B-10. State diagram for a user account (UC-7, UC-8, UC-9).

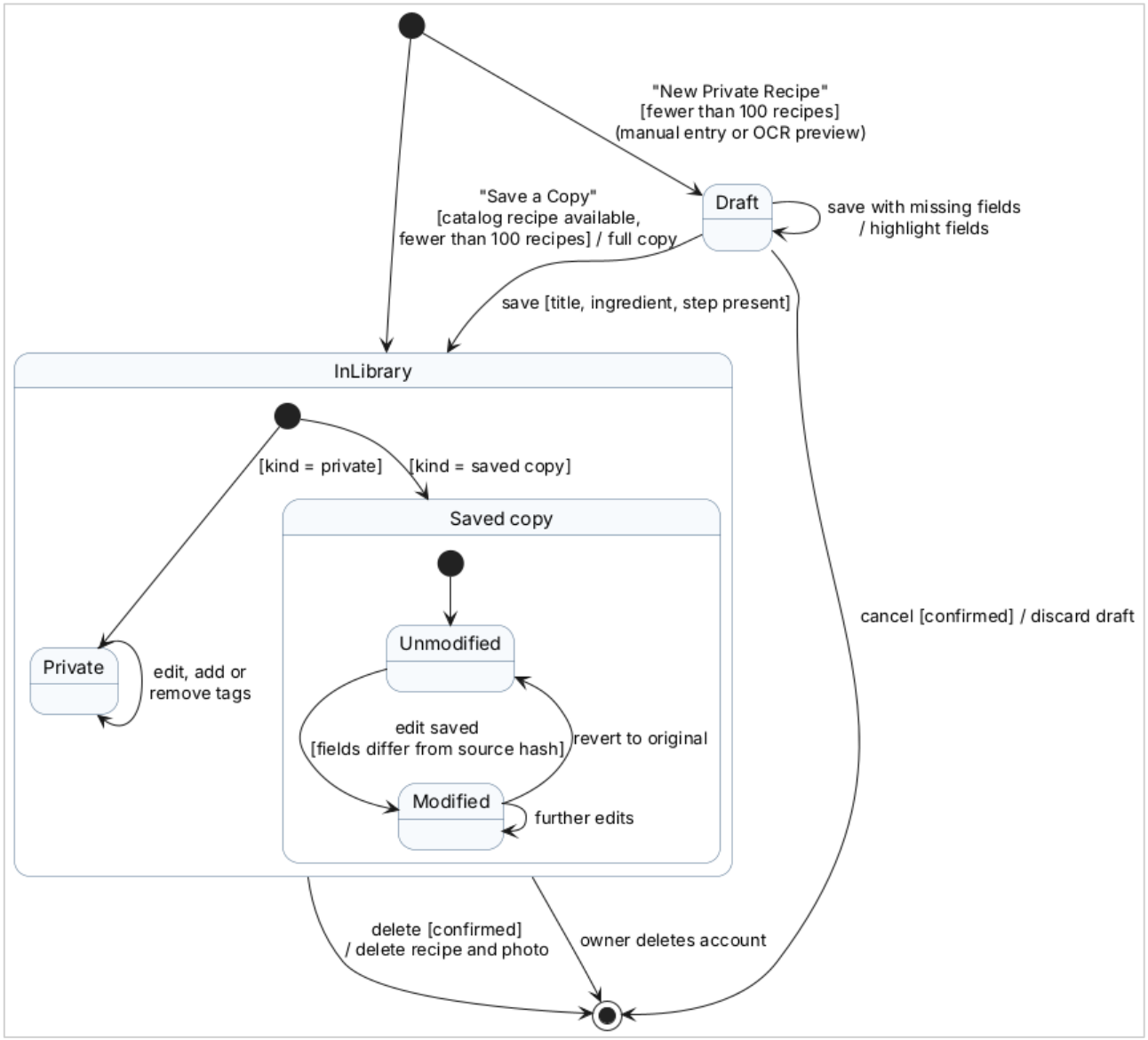


Figure B-11. State diagram for a recipe in a user's Library, including the 100-recipe limit (UC-2, UC-3, UC-4, UC-5).

Appendix C: To Be Determined List

1. OCR Engine. Resolved in v1.5: Tesseract.js in the user's browser (§6.1).
2. Database Engine. Resolved in v1.1: PostgreSQL 18 (§6.1).
3. Hosting provider. Resolved in v1.5: Vercel Hobby plan with Neon's free PostgreSQL, $0 (§6.1).
4. Email delivery service. Resolved in v1.4: Resend free plan, $0 (§6.1).
5. Programming language and web framework. Resolved in v1.5: TypeScript with Next.js (§6.1).
6. Numeric targets added in v1.1 (the NFRs with Source "Team" in §5, the 100-recipe limit, and the retention periods) to be confirmed with the product owner.
7. GitHub storage needed for 7 nightly database backups, to be checked against the free plan.
8. The seed recipe list and the sources of its photos (BR-12).

Appendix D: Elicitation, Validation, AI Disclosure

D.1: Elicitation Log

Method: interviewing, with an AI playing the client. Role prompt: "You are Dana Reyes, the product owner for Forkful, a web app that lets home cooks save recipes from a public catalog, edit saved copies, and add or scan private recipes. Answer as the client, in plain language, and stay in role." The client's answers were generated by Claude (see D.3)

1. **Zero-shot prompt**
   Interviewer: What should a home cook be able to do with Forkful, and who else needs something from it?
   Client: Cooks should browser and search the public catalog without an account, save a copy of any recipe and edit the copy without touching the original, add their own recipes by typing or scanning a printed card, keep those private, and delete anything they own at any time. The system administrator needs to pause and restart the parser and OCR jobs without kicking users off, and the content curator needs to keep the catalog clean.

2. **Few-shot prompt (k = 2)**
   Interviewer: Here are two examples from a library app. Q: What should happen when a patron places a hold? A: The system records the hold for that patron only and confirms it; the book's catalog record is not changed. Q: How should removing a hold work? A: The system asks the patron to confirm, then removes only that patron's hold. Now answer in the same style: what should Forkful do when a user browses the catalog, saves a copy, edits it, adds a private recipe, and deletes recipes?
   Client: The catalog shows 20 recipes per page, and anyone can filter by cuisine and tag. Each recipe shows its title, ingredients, and steps. Saving a copy creates a new recipe owned by the user and leaves the catalog record unchanged. The user can edit any field of the copy, and a "Modified" indicator shows when it differs from the original. A private recipe needs a title, ingredients, and steps, with optional tags, and only its owner can see it. Deleting asks for confirmation first, removes only the user's own record, and never touches the catalog original.

3. **Chain-of-thought prompt**
   Interviewer: Think step by step about how long a person will wait in each case, then give me limits for catalog search and for scanning a recipe with OCR.
   Client: Searching is something people do repeatedly while planning dinner, so a long wait makes them give up. I'd cap it at 4 seconds for a catalog of up to 50,000 recipes under normal load. A scan happens once per recipe and the user expects some processing, so they'll accept longer: 15 seconds at the 90th percentile for a single 300-DPI image. Uploads can be JPG, PNG, or PDF up to 10 MB. After the scan, show the extracted text in an editable field next to the image, and save the corrected result as a private recipe.

4. **Zero-shot follow-up (accounts and profiles)**
   Interviewer: What do you need for sign-up and for managing an account?
   Client: Sign-up takes a username, email, and password. Passwords need at least 8 characters, usernames and emails must be unique, and the person must confirm they are at least 13 because of COPPA. We email a verification link, which expires after 24 hours and can be re-sent, and nobody logs in until the email is verified. On the profile page they see their username and email and can change their username and password. A new email must be verified before it replaces the old one,

and the current password is required to change the email or password. They can delete their account after confirming, which removes everything they own and logs them out.

5. **Zero-shot follow-up (administration, security, feedback)**
   Interviewer: What does the administrator need, and what are your security and usability expectations?
   Client: The admin needs a console to view and manage accounts, and to pause, resume, or restart the parser and OCR jobs without disrupting active sessions. Every admin action must be logged with who did it, when, and what. User data must be encrypted, and access must be role-based so cooks only reach their own recipes. Finally, the site should always tell users whether an action worked or not.

D.2: Validation

Problem Found	Issue	Fix
"Catalog searching should be fast"	SMART: Measurable. "Quickly" can't be measured, so a tester can pass or fail it.	Rewrote to "Done in 4 seconds for a catalog of up to 50,000 recipes under normal load" (REQ-1.2, NFR-1).
"OCR time applies to a certain percentile of scans"	Ambiguity Trap, SMART: Specific. The percentile and input conditions were unstated, so it could be read several ways	Rewrote to "OCR should take 15 seconds with a 90% accuracy for a 300-DPI image" (REQ-4.2).
Requirement stating "support all file types" for OCR	Stakeholder intent	Narrowed down to "PNG, JPG, and PDF files up to 10MB" being supported (REQ-4.1)
REQ-4.2 (90th percentile") and NFR-2 (90% accuracy) disagreed	Ambiguity Trap	NFR-2 now covers time only; accuracy moved to NFR-21.
NFR-3: Encrypt user data based on current standards	SMART: Measurable	Rewrote to TLS 1.2 or newer in transit and AES-256 at rest.
Catalog described as an outside API (§2.8, §3.3) but drawn as a schema in Forkful's database (Figure 2-1)	Consistency	Catalog is a schema read through the internal Catalog Service (§2.8, §3.3, §6.1).
Server OCR, stored images, and a scheduled crawler could not run on free hosting	Feasibility (cost)	Moved OCR to the browser, dropped stored images and PDF input, and replaced the crawler with import by URL (§6.1, §6.5).
The full feature set was too large to build and host as a class project	Feasibility (scope)	Kept the core features in v1.0 and moved the extras to Appendix E, Potential Future Features (§1.2, §6.1).

D.3: AI Disclosure (H/A/X)

Where AI was used	Tag
Elicitation: AI stakeholder interview (Claude played as the client, prompts written by team)	A
REQ-1.x to 3.x and 5.x written by team, adapted from public example software requirements	H
REQ-4.1 to 4.4 drafted with Google AI Overview	A
Use case tables UC-1 and UC-2 (§4.2.2, §4.4.2)	A
Use case diagram (Appendix B)	A
Tabular use cases UC-3 to UC-13; NFR-6 to NFR-31; BR-4 to BR-6; §3.6; and §6.	A
v1.4 diagrams: Figures B-1, B-3, and B-5 to B-11 written in PlantUML drafted with Claude and reviewed by team.	A
v1.5 Appendix E (Potential Future Features), removal of deferred requirements, renumbering, and updated figures, drafted with Claude and reviewed by the team	A

Google AI Overview was also used in tandem to provide clarification on details regarding OCR, Databases, Communication Interfaces, and Operations.

Appendix E: Potential Future Features

Features the team considered but left out of the class-project build to keep it small and free to host. Each could be added later as its own feature in §4, with new REQ and NFR numbers.

Feature	What it would add	Why it is not in v1.0	What it would change
Trash and restore	Deleted recipes wait 30 days in a Trash and can be restored before they are permanently deleted.	Needs a daily purge job and restore rules that respect the 100-recipe limit.	UC-5, BR-10, the daily cleanup job (§6.1)
Favorites	Mark recipes as favorites and filter the Library to show only favorites.	Nice to have; tags already cover grouping.	UC-10, recipe table
Collections	User-named groups of recipes (the “sets” in the original idea); one recipe can be in several.	Adds two tables and extra screens.	UC-10, data model (§6.2)
Content Curator role	A separate role for catalog upkeep: merge duplicate entries and re-import failed ones.	The System Admin can do this alone at class-project size.	UC-11, roles, BR-7
Account lockout	Lock an account for 15 minutes after 5 failed logins and email the owner.	Extra security work beyond a class demo.	UC-9, users table
Audit log	Record every admin action with who, when, and what.	Needs its own table and viewing screen.	UC-6, UC-11, data model

Download my data	Let users download their profile and recipes as a JSON file.	Not needed for the demo; account deletion already covers the privacy basics.	UC-7, §2.4
Revert and compare	Revert a saved copy to the current catalog original, or view the two side by side.	The “Modified” indicator covers the core need.	UC-2
Automatic catalog updates	A scheduled crawler that adds and refreshes recipes from cooking sites.	Needs an always-on server and raises rate-limit and terms-of-service risks.	UC-12, hosting (§6.1)
Higher limits or a paid tier	More recipes and photos per user, possibly as a subscription.	Vercel’s free plan is non-commercial (BR-11), and payments need a payment provider.	BR-10, BR-11, hosting (§6.1)
Better handwriting OCR	Use a cloud OCR service for handwritten recipe cards.	Cloud OCR costs money per page and sends private images to a third party.	UC-4, §6.1
Volume-to-weight conversion	Convert between cups and grams (for example, 1 cup flour to 120 g).	Needs a density for every ingredient, since a cup of flour and a cup of sugar weigh different amounts.	§4.1, ingredient data